



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECP2523: DATABASE

SEM I 2025/ 2026

DATABASE DESIGN DESCRIPTIONS: CONCEPTUAL DATABASE DESIGN FOR HASTA CAR RENTAL SYSTEM

Group Name: *Gimme5*

Members:

1. Gwee Zi Ni - A24CS0078 (Team Leader)
2. Kalaitharan A/L Palanyvelu - A24CS0091
3. Lee Jia Yee - A24CS0260
4. Michelle Ho Chia Xin - A24CS0110
5. Tay Xin Ying - A24CS0200

Stakeholder: Hasta Travel & Tours (Sdn. Bhd.)

Printing Date: 3/2/2026

To ensure legibility, high-resolution versions of all figures and diagrams included in this report can be accessed via this [Google Drive link](#):  [High-Resolution Report Photos](#)

Table of Content

1.0 Introduction.....	2
2.0 DFD (to-be).....	3
3.0 Data & Transaction requirement.....	4
3.1 Proposed business rule.....	4
3.2 Proposed data & transactional.....	7
4.0 Database conceptual design.....	9
4.1 Conceptual ERD.....	9
4.2 Enhanced ERD (EERD).....	12
5.0 Data Design.....	13
5.1 Data Description.....	13
5.2 Data Dictionary.....	15
5.2.1 Entity: bookings.....	15
5.5.2 Entity: claims.....	17
5.5.3 Entity: colleges.....	18
5.5.4 Entity: faculties.....	18
5.5.5 Entity: feedback.....	19
5.5.6 Entity: fines.....	19
5.5.7 Entity: inspections.....	20
5.5.8 Entity: loyalty_cards.....	21
5.5.9 Entity: maintenance_logs.....	22
5.5.10 Entity: notifications.....	22
5.5.11 Entity: password_reset_tokens.....	23
5.5.12 Entity: payments.....	23
5.5.13 Entity: pricing_rules.....	24
5.5.14 Entity: pricing_tiers.....	24
5.5.15 Entity: promotions.....	25
5.5.16 Entity: users.....	25
5.5.17 Entity: user_vouchers.....	27
5.5.18 Entity: vehicles.....	27
5.5.19 Entity: vehicle_types.....	29
5.5.20 Entity: vouchers.....	30
6.0 Summary.....	31
7.0 Appendice.....	32
7.1 Use Case Diagram for the System.....	32
7.2 Activity Diagram for the System.....	35
7.3 Meeting Log.....	50
Bibliography.....	52

1.0 Introduction

Phase 2 focuses on the database conceptual design for the Hasta Car Rental System based on the requirements identified in Phase 1. The purpose of this phase is to translate the business processes and user requirements into a structured database design that can support daily car rental operations effectively.

In this phase, the main entities, attributes, and relationships are identified according to the proposed business rules. The relationships between entities, including their multiplicities, are analysed to ensure accurate representation of real-world operations such as booking, vehicle usage, payment, and customer management. A conceptual Entity Relationship Diagram (ERD) and an Enhanced ERD (EERD) are developed using UML Notation to model the data structure clearly.

Additionally, data and transaction requirements are examined to ensure that the database design supports essential operations such as data entry, data updates, and data retrieval. A data dictionary is also prepared to define each entity and attribute in detail. This ensures consistency and clarity throughout the database design.

Overall, this phase provides a structured foundation for the logical and physical database design by ensuring that all data requirements are accurately captured and aligned with the system's business processes. This phase focuses on translating the requirements identified in Phase 1 into a database conceptual design, including Data Flow Diagrams (DFD), Entity-Relationship Diagrams (ERD), Enhanced ERD (EERD), and a data dictionary.

2.0 DFD (To-Be)

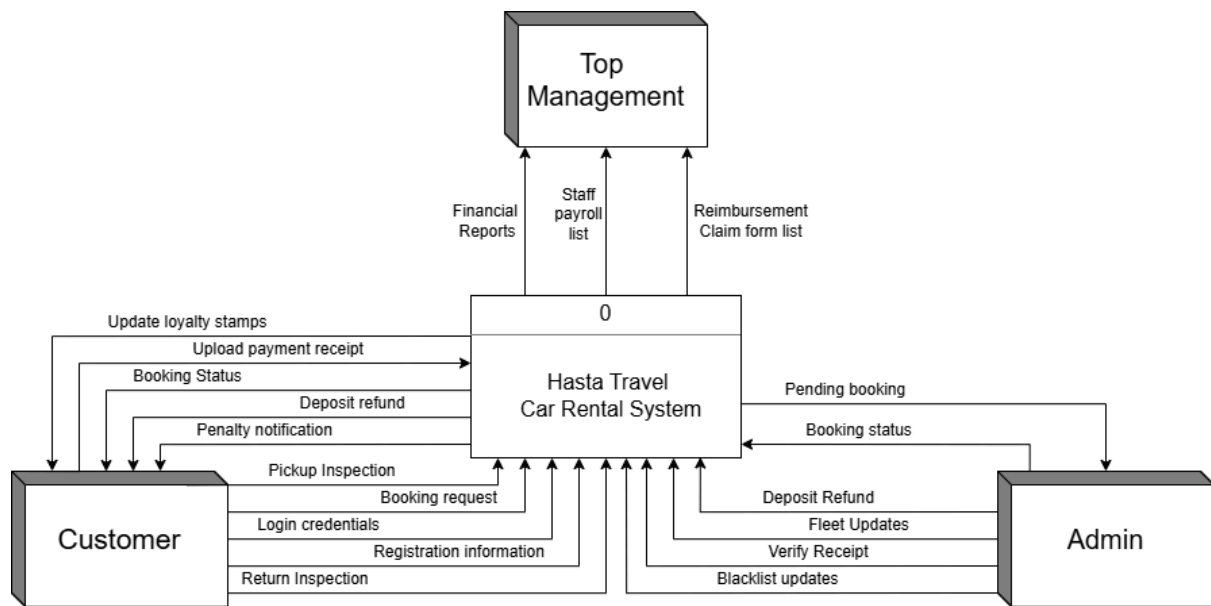


Figure 2.0.1: Context Diagram for DFD (TO-BE system)

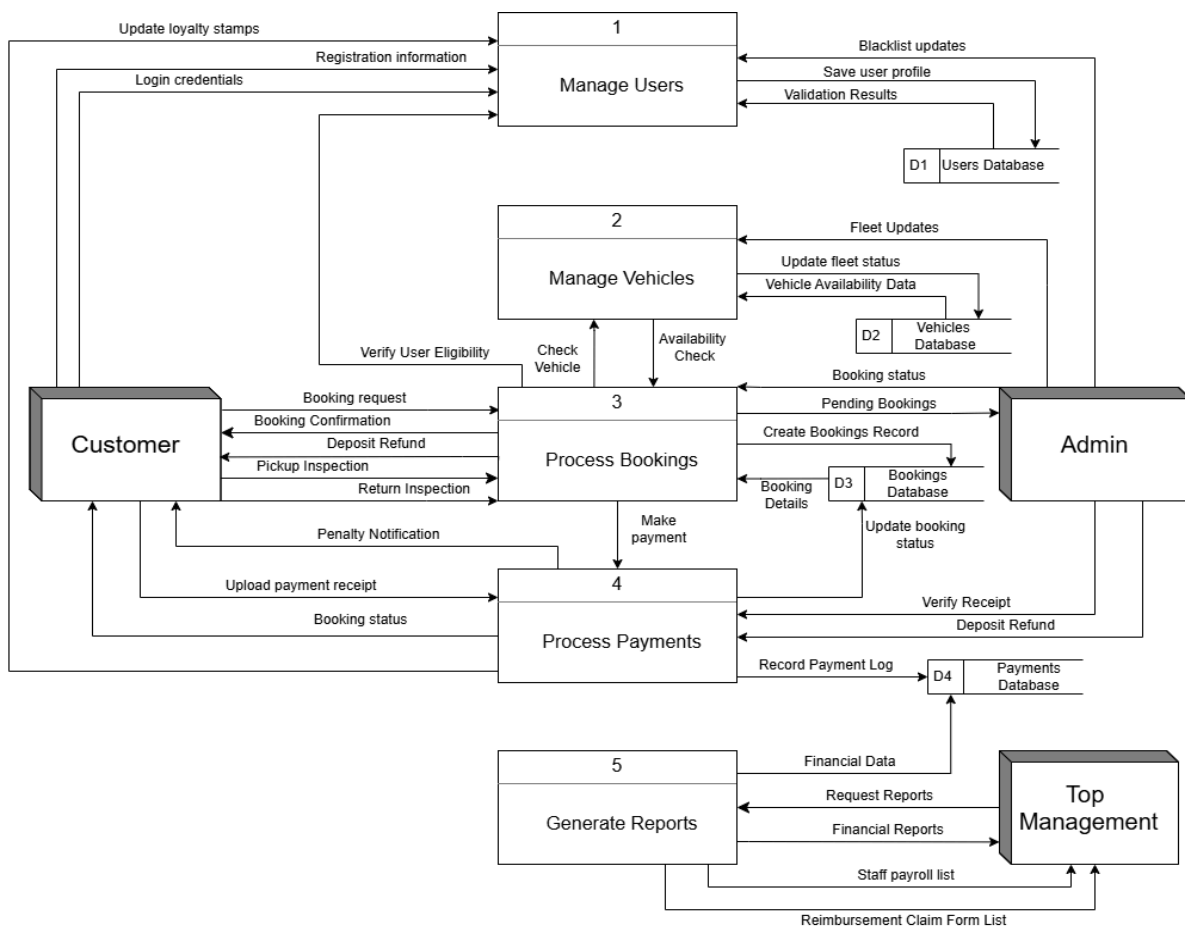


Figure 2.0.2: Level 0 Diagram for DFD (TO-BE system)

3.0 Data & Transaction requirement

3.1 Proposed business rule

This section describes the proposed business rules derived from the current and intended business processes. These rules define the constraints that govern how data is created, updated, and related within the database.

3.1.1 Operational and Policy-Based Rule

This subsection outlines the non-technical constraints derived from Hasta Car Rental's standard operating procedures. These rules focus on user eligibility, rental conditions, and financial policies that the system must enforce to ensure compliance with company regulations.

1. Customer Eligibility & Verification

UTM Exclusivity: The system offers services strictly to UTM members only. A valid Matric Number (for students) or Staff ID is a mandatory unique identifier during registration.

Document Verification: An account cannot be successfully registered until the customer has uploaded three mandatory documents: IC/Passport, Driving License, and Matric ID/ Staff ID.

2. Booking & Reservation Constraints

Advance Notice: To allow for vehicle preparation, the system enforces a strict rule that bookings must be made at least 12 hours in advance of the pickup time.

Minimum Duration: The system enforces a minimum rental duration of 1 hour for all bookings.

Availability Integrity: The system automatically prevents a vehicle from being booked if its status is "Approved", "Rented", or "Maintenance" during the selected date range. Any overlapping time slots are strictly rejected to prevent double-booking.

3. Payment & Deposit Logic

Tiered Deposit: The system automatically calculates the deposit based on rental duration:

Rentals < 15 Days: A standard deposit of RM 50 is applied.

Rentals $\geq$ 15 Days: The deposit amount is equal to the Total Rental Fee (100% of rental price).

Proof-of-Payment: A booking is only considered "Active/Waiting Verification" after the user uploads a valid receipt file. Bookings without receipt uploads remain in a pending state.

4. Return & Penalty Calculations

Manual Penalty Issuance: Late return penalties are calculated based on predefined rules but are manually confirmed and recorded by staff in the system due to varying circumstances.

5. Loyalty Program Automation

Stamp Accumulation: The system automates loyalty tracking by awarding 1 stamp for every completed booking where the rental duration is 3 hours or more. This calculation happens automatically when the admin approves the payment/completion of the booking.

6. **Reward Milestones:** The system tracks cumulative stamps and unlocks rewards at specific tiers (e.g., 3, 6, 9 stamps), automatically generating a digital voucher code for the user once a threshold is reached.

3.1.2 Data-Oriented Business Rule (Structural Constraints)

This subsection defines the structural constraints and relationships (cardinality) enforced by the database schema to maintain data integrity.

User Entity Constraints

1. Each user must be registered in the system before making any booking.
2. A user is identified by a unique user ID and can have one or more bookings over time.
3. A user who is marked as blacklisted is not allowed to make new bookings until the blacklist status is removed by an administrator.

Booking & Scheduling Constraints

1. Each booking must be associated with exactly one vehicle, while a vehicle may be associated with multiple bookings at different times.
2. A vehicle cannot be booked by more than one user for overlapping rental periods.
3. Each booking must have a valid pickup date and time, return date and time, where the return time must be at least one hour after the pickup time.
4. Each booking is created with an initial status (e.g., pending) and may later be updated to approved, rejected, modified, or cancelled.

Payment & Financial Constraints

1. Each booking must be associated with at least one payment record for rental fees.

2. Payment records must include payment amount, payment method, receipt upload, and verification status, and must be linked to a specific booking ID.

Inventory & Maintenance Constraints

1. Vehicle availability status must be updated automatically when a booking is confirmed, cancelled, or when a vehicle is under maintenance.
2. Each vehicle may have multiple maintenance records, but each maintenance record refers to only one vehicle.

Claims & Loyalty Constraints

1. Staff members can submit claims for reimbursement, and each claim must be reviewed and approved or rejected by top management.
2. Loyalty rewards are granted to users only after a completed booking that meets the minimum rental duration requirement.

3.2 Proposed data & transactional

This section outlines the specific data entities the system must store and the transactional operations that manipulate this data to fulfill the business rules.

3.2.1. Data Requirements (Entities)

The database must be designed to store the following core entities and their attributes:

User Data:

- Stores personal details: Name, Email, Phone, NRIC/Passport.
- Stores authentication & verification details: Matric/Staff ID, Password, Account Role (Student/Staff/Admin), and Document Paths (License, Matrix Card).
- Stores loyalty status: Current Stamp Count and accumulated Rewards.

Vehicle Data:

- Stores physical attributes: Brand, Model, Year, Color, Plate Number, Capacity, Transmission Type.
- Stores operational status: Availability Status (Available/Rented/Maintenance), Pricing Tier, and Fuel Level.
- Stores compliance data: Road Tax Expiry, Insurance Expiry, and owner document paths (Grant/Geran).

Booking Transaction Data:

- Stores schedule details: Pickup Date/Time, Return Date/Time, Pickup/Drop-off Locations.
- Stores financial details: Total Rental Fee, Security Deposit Amount, Refund Bank Details, and applied Voucher/Promo Codes.
- Stores status flow: Booking Status (e.g., Waiting for Verification, Approved, Completed, Cancelled).

Inspection & Condition Data:

- Stores inspection photos: JSON paths for photos (Front, Back, Left, Right, Dashboard, Keys).
- Stores checklist data: Fuel Level (Bars), Mileage, and Damage descriptions.

Financial Data (Payments & Fines):

- Stores transaction proof: Payment Receipt Image, Verified By (Staff ID), and Timestamp.
- Stores penalties: Fine Amount, Reason (e.g., "Fuel Discrepancy"), and Payment Status (Paid/Unpaid).

3.2.2 Transactional Requirements (Operations)

The system must support the following key transactions:

Account Management:

- Registration: Create a new user record awaiting verification.
- Profile Update: specific update of phone, address, or password.

Rental Operations:

- Check Availability: Query database for vehicles where status = Available AND no overlapping bookings exist for the requested dates.
- Create Booking: Insert a new booking record, reserve the vehicle slot, and debit the voucher usage (if applicable).
- Cancel Booking: Update booking status to 'Cancelled', release the vehicle schedule, and revert voucher usage/stamps.

Fleet Control:

- Update Vehicle Status: Manually set status to 'Unavailable' or automatically set to 'Unavailable' if documents expire.
- Log Maintenance: Insert a maintenance record and deduct cost from financial reporting.

Processing & Verification:

- Verify Payment: Update Booking to 'Approved' and trigger Loyalty Stamp increment if duration > 3 hours.
- Process Inspection: Compare 'Pickup' and 'Return' fuel levels; automatically generate a Fine record if Return Fuel < Pickup Fuel.
- Refund Deposit: Update Deposit Status to 'Returned' after confirming no outstanding fines exist and the staff has attached the receipt of refund.

4.0 Database conceptual design

The conceptual design phase focuses on identifying the entities, attributes, and relationships required to support the Hasta Car Rental System's functionality. This design is based on the requirements gathered in Phase 1, ensuring that the database supports key operations such as user management, vehicles inventory, booking processing and voucher logics. The multiplicity of each relationship is determined based on the proposed business rules and is represented using UML notation to reflect real-world constraints.

4.1 Conceptual ERD

According to the data modeling guidelines outlined by [Visual Paradigm Online](#) (n.d.), a conceptual data model focuses on identifying key entities and relationships at a high level, without including detailed attributes.

Based on this approach, the conceptual model in this project intentionally excludes attributes and serves to illustrate the overall structure of the system. Attribute details, keys, and constraints are subsequently defined in the logical and physical data models to maintain clarity and proper separation of concerns.

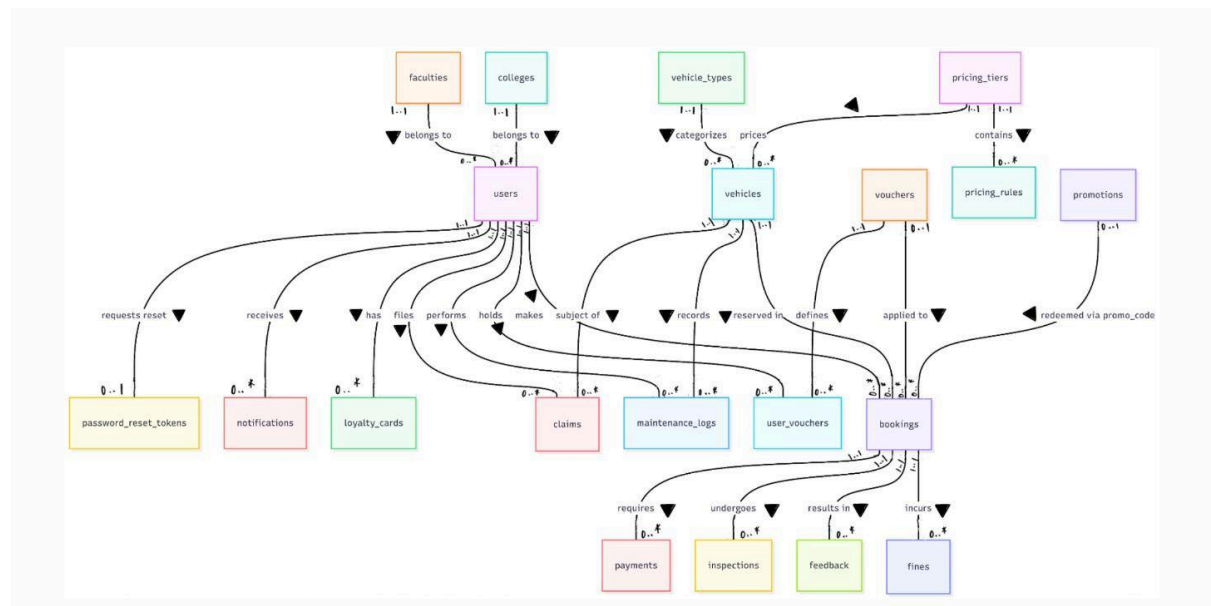


Figure 4.1.1: Conceptual ERD

Relationship Definitions:

1. User Management (Single Table Strategy):

- Concept: The system manages distinct user roles using a Single Table Inheritance strategy. All user data is stored in the users entity to optimize authentication performance.

- Roles: The system defines three roles via the role attributes:
 - Customer: Primarily students (linked via college_id and faculty_id).
 - Admin: Operational staff who manage daily tasks (bookings, fleet).
 - Top Management: Strategic users with access to financial reports and sensitive data.
- Differentiation: The role attribute discriminates between these types. Student-specific attributes(e.g., matric_card_path) are nullable fields that are only populated when the role is 'customer'.
- Relationship: A 'customer' user belongs to one college and one faculty. Admin and Top Management users effectively bypass these academic relationships.

2. Booking Lifecycle:

- Core Flow: A booking serves as the central transaction entity. It links a specific user(customer) to a specific vehicle for a defined time range.
- Dependencies: The booking is the parent entity for payments, inspections (pickup/return checks), fines, and feedback. These entities are operationally dependent on the existence of a valid booking ID.
- Constraints: A booking is considered "Confirmed" primarily upon payment verification.

3. Vehicle Inventory & Pricing:

- Categorization: Vehicles are categorized by vehicle types (e.g., Sedan, Axia) and assigned a pricing tier.
- Ownership: The system supports a hybrid fleet. Vehicles can be owned by Hasta or by third-party owners. Third-party ownership details (e.g., owner_name, owner_ic_path) are stored directly within the vehicle table attributes rather than a separate entity.
- Pricing Logic: A pricing_tier contains multiple pricing_rules (not rates). These rules define the specific constraints (e.g., hour_limit) and the corresponding cost (price), allowing for tiered pricing logic (e.g., "First hour: RM30" vs "Daily: RM200").

4. Promotions & Vouchers:

- Vouchers (Strict Referential Integrity): A Booking can optionally be linked to a specific voucher_id. This occurs when a user redeems a personal voucher

from their collected vouchers (stored in the `user_vouchers` pivot table), ensuring the voucher is marked as used effectively.

- Promotions (Logical Association): Promotions are linked to bookings logically via the `promo_code` attribute. This allows multiple bookings to reference a generic marketing code (e.g., "MERDEKA2025") without requiring a strict foreign key constraint payload for every simple campaign.

5. Loyalty & Notifications:

- Loyalty: A `loyalty_card` is strictly 1-to-1 with a user. It tracks rental milestones (via `unread_stamps`), which the system uses to automatically generate `user_vouchers` as rewards.
- Notifications: The `notifications` entity stores system alerts (e.g., booking approvals, fine issuance) and is directly linked to the recipient user.

6. Fleet Maintenance

- Audit Trail: The `maintenance_logs` entity serves as an audit trail for vehicle upkeep (cost, description).
- Accountability: It creates a three-way link between the vehicle (subject of maintenance), the user (staff member performing nature), and the maintenance details.

4.2 Enhanced ERD (EERD)

Enhanced ERD features are applied in this design to better represent real-world requirements. Specialisation is used to distinguish different user roles such as students, staff, and administrators, while maintaining a unified user structure. These enhancements improve data organisation and reduce redundancy.

According to the data modeling guidelines outlined by [Visual Paradigm Online](#) (n.d.), a conceptual data model focuses on identifying key entities and relationships at a high level, without including detailed attributes.

Based on this approach, the conceptual model in this project intentionally excludes attributes and serves to illustrate the overall structure of the system. Attribute details, keys, and constraints are subsequently defined in the logical and physical data models to maintain clarity and proper separation of concerns.

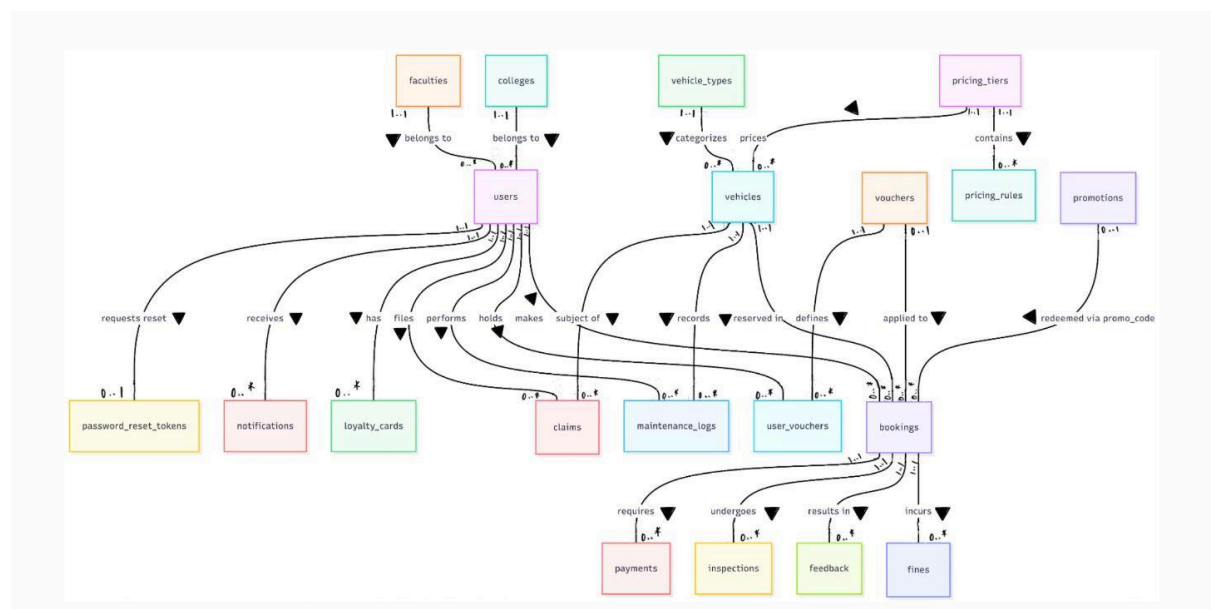


Figure 4.2.1: Enhanced ERD (EERD)

5.0 Data Design

5.1 Data Description

Database is significant and can be identified as the basis of a system to store and support the smooth and well integrated system process. It consists of lots of tables that include information domains and represent the major entities. Each table has its primary key as their unique. The relationship between entities is maintained by using foreign keys. This ensures data integrity and supports the processing logic of the system. In our project, the major data or systems entities are stored into a relational database named carrentalsystem, processed and organized into 26 entities as listed in Table 5.1.

Table 5.1.1: Description of Entities in the Database

No.	Entity Name	Description
1	bookings	Records rental details, including pickup and return schedules, payment proofs and the approval status
2	claims	Records the financial claims for the admin staff for delivery tasks, fuel, or other reimbursement
3	colleges	Records customers with their corresponding colleges so that can be collected as the data to visualize at the admin or top manager dashboard
4	faculties	Records customers with their corresponding faculties so that can be collected as the data to visualize at the admin or top manager dashboard
5	feedback	Stores customers feedbacks, ratings and review regarding vehicle condition after a booking
6	finances	Records penalties issued to specific bookings, including the fine amount, reason, evidence receipts, and payment status
7	inspections	Stores vehicle inspection records (checklist and photos) conducted before and after a rental to track vehicle condition
8	loyalty_cards	Tracks user loyalty participation by storing the count of accumulated stamps earned

		based on business rules which is receiving 1 stamp for rentals exceeding 3 hours for reward redemption
9	maintenance_logs	Records the history of vehicle maintenance, including dates, costs, as well as descriptions of service or repairs
10	notifications	Stores system notifications sent to users, including the message content and read status
11	password_reset_tokens	Stores the temporary OTP codes generated for password recovery, allowing the system to verify user identity and enforce expiration limits 15 minutes
12	payments	Records transaction details including amount, payment method, uploaded receipt images and verification status
13	pricing_rules	Stores the price list for specific time blocks (like 1 hour,12 hours, 24 hours) for every pricing tier
14	pricing_tiers	Groups vehicles into tier categories so that prices can be managed collectively
15	promotions	Stores details of stamp collected and their corresponding rewards and display on the system to attract users
16	users	Stores account information for all system users, including login credentials and profile data.
17	user_vouchers	A bridge table that links specific vouchers to the users who have claimed or used them
18	vehicles	Stores detailed information about the cars available for rent, including plate number, model, color, price rate, and availability status
19	vehicle_types	Categorizes vehicles into types such as Sedan, Compact and Hatchback to provide the filter features for customers
20	vouchers	Defines redeemable vouchers with specific codes, values and usage limits for users

5.2 Data Dictionary

5.2.1 Entity: bookings

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for each booking transaction.
user_id	bigint	Foreign key. References the users table to identify the account holder who made the reservation
vehicle_id	bigint	Foreign key. References the vehicles table to identify the specific car being rented
pickup_date_time	datetime	Scheduled start date and time of the rental. Used to calculate the total rental duration.
return_date_time	datetime	Scheduled end date and time. Must be strictly later than the pick_up_date time.
pickup_location	varchar(50)	General location name for pickup
custom_pickup_address	text	Specific address details provided by the user when pickup location is not Student Mall as default
dropoff_location	varchar(50)	General location name for dropoff
customer_name	varchar(50)	Name of the person driving
customer_phone	varchar(20)	Contact number for immediate communication regarding the booking
emergency_contact_name	varchar(100)	Name of the emergency contact person
emergency_contact_phone	varchar(20)	Contact number of the emergency contact person
emergency_relationship	varchar(20)	Relationship to emergency contact to the customer
license_image	varchar(255)	Stores the file path or URL of the uploaded driving license image for verification purposes.

payment_receipt	varchar(255)	Stores the file path of the payment proof uploaded by the user.
custom_dropoff_address	text	Specific address details provided by the user when dropoff location is not Student Mall as default
total_rental_fee	decimal(8, 2)	Total calculated cost of the rental period before any discounts
deposit_amount	decimal(8, 2)	Security deposit amount collected to cover potential damages or fines
promo_code	varchar(255)	The specific promo code string applied to this booking,if any.
status	varchar(255)	Current state of booking. Values: 'Pending', 'Cancelled', 'Returning', 'Completed'
deposit_status	varchar(20)	Current status of the security deposit.
deposit_receipt_path	varchar(255)	Path to deposit refund receipt
deposit_returned_at	timestamp	Time when deposit was returned
rejection_reason	text	Reason if booking was rejected
payment_verified	tinyint(1)	Flag to indicate if the admin has verified the payment receipt (1 = Verified, 0 = Pending).
voucher_id	bigint	Foreign key. Reference vouchers table and links to the specific voucher used for discount tracking. Nullable
processed_by	bigint	Foreign Key. Identifies the Admin who approved or rejected the booking. Nullable because it remains empty (NULL) when the booking is first created and only gets populated once an Admin takes action.
refund_bank_name	varchar(60)	Name of the user's bank such as 'Maybank' to facilitate deposit refunds
refund_account_number	varchar(50)	User's bank account number for deposit refund transfer

refund_recipient_name	varchar(100)	Name of the account holder for the refund transaction
created_at	timestamp	Timestamp when the booking request was first created
updated_at	timestamp	Timestamp when the booking details or status were last modified

5.5.2 Entity: claims

Attribute Name	Type	Description
id	bigint	Primary key.Unique auto incrementation identifier for each claim records
matric_staff_id	varchar(15)	The staff ID of the individual making the claim.
user_id	bigint	Foreign key. References the users table to identify the claimant
claim_type	varchar(20)	The category/type of the claim (e.g., Fuel Reimbursement, Maintenance, Accident).
vehicle_plate	varchar(10)	The license plate number of the vehicle involved in the claim.
amount	decimal(10,2)	The total monetary value being claimed.
claim_date_time	datetime	The specific date and time when the incident or expense occurred.
description	text	Detailed explanation or notes regarding the reason for the claim.
status	enum('Pending', 'Approved', 'Rejected')	Current status of the claim processing (e.g., 'Pending', 'Approved', 'Rejected').
action_reason	text	The additional remark provided by the top manager for approving or rejecting the claim.
receipt_path	varchar(255)	Stores the file path or URL to the uploaded receipt image serving as proof of expense.

payment_proof	varchar(255)	Stores the file path or URL to the proof of payment (reimbursement) uploaded by the top manager.
processed_by	bigint	Foreign key. References the users table to identify who processes the claim. Nullable because it remains empty(NULL) when the claim is first created.
processed_at	timestamp	Timestamp indicating when the claim was processed.
created_at	timestamp	Timestamp when the claim record was created.
updated_at	timestamp	Timestamp when the claim record was last modified.

5.5.3 Entity: colleges

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for each college
name	varchar(50)	The name of the college (e.g., KTDI, KTR).
created_at	timestamp	Timestamp indicating when the college record was created.
updated_at	timestamp	Timestamp indicating when the college record was last modified.

5.5.4 Entity: faculties

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for each faculty
name	varchar(100)	The official name of the faculty (e.g., Faculty of Computing).
created_at	timestamp	Timestamp indicating when the faculty record was created.
updated_at	timestamp	Timestamp indicating when the faculty record was last modified.

5.5.5 Entity: feedback

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for feedback record.
booking_id	bigint	Foreign key. References the bookings table to identify the rental transaction
ratings	json	JSON data storing detailed rating metrics (e.g., cleanliness score, vehicle condition score).
category	varchar(50)	The classification of the feedback (e.g., Mechanical Issue, Aircond Issue). Nullable.
description	text	Textual comment providing full details of the user's feedback or experience.
return_photo_url	varchar(255)	Stores the file path or URL of the photo taken upon vehicle return, used as evidence for vehicle condition.
created_at	timestamp	Timestamp indicating when the feedback was submitted.
updated_at	timestamp	Timestamp indicating when the feedback record was last modified.

5.5.6 Entity: fines

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for each fine.
booking_id	bigint	Foreign key. References the bookings table to identify the associated rental transaction.
amount	decimal(8, 2)	The monetary amount charged for the fine.
receipt_path	varchar(255)	Stores the file path or URL to the uploaded receipt showing proof that the fine has been paid.

reason	varchar(255)	The justification for issuing the fine (e.g., 'Late Return', 'Vehicle Damage', 'Smoking').
status	varchar(30)	The payment status of the fine (e.g., 'Unpaid', 'Paid').
paid_at	timestamp	Timestamp indicating when the fine payment was completed.
created_at	timestamp	Timestamp indicating when the fine record was created.
updated_at	timestamp	Timestamp indicating the last time the fine status or details were modified.

5.5.7 Entity: inspections

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for each inspection.
booking_id	bigint	Foreign key. References the bookings table to identify the associated rental transaction.
type	enum('pickup', 'return')	The stage of inspection, either 'pickup' (before rental) or 'return' (after rental).
fuel_level	int	The fuel gauge reading at the time of inspection (e.g., number of bars, 0-10).
mileage	int	The vehicle's odometer reading (total distance traveled) at the time of inspection.
remarks	text	General notes or comments made during the inspection.
photo_front	varchar(255)	Stores the file path or URL to the photo of the vehicle's front view.
photo_back	varchar(255)	Stores the file path or URL to the photo of the vehicle's rear view.
photo_left	varchar(255)	Stores the file path or URL to the photo of the vehicle's left side.

photo_right	varchar(255)	Stores the file path or URL to the photo of the vehicle's right side.
photo_dashboard	varchar(255)	Stores the file path or URL to the photo of the dashboard (fuel/mileage).
photo_keys	varchar(255)	Stores the file path or URL to the photo of the vehicle keys (usually for return inspection).
damage_photos	json	JSON array storing file paths/URLs to photos of specific damages found on the vehicle.
damage_description	text	Textual details describing any new or existing damage found on the vehicle.
created_by	bigint	Foreign key. References the users table to identify who performed the inspection.
created_at	timestamp	Timestamp indicating when the inspection was recorded.
updated_at	timestamp	Timestamp indicating when the inspection record was last modified.

5.5.8 Entity: loyalty_cards

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for each loyalty card
user_id	bigint	Foreign key. References the users table to identify the cardholder.
stamps	int	Current count of loyalty stamps accumulated by the user.
unread_stamps	int	Counter for new stamps that the user hasn't seen yet (used for notification badges/popups).
created_at	timestamp	Timestamp indicating when the loyalty card was first issued.

updated_at	timestamp	Timestamp indicating when the card details (e.g., stamp count) were last updated.
------------	-----------	---

5.5.9 Entity: maintenance_logs

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for each maintenance log.
vehicle_id	bigint	Foreign key. References the vehicles table to identify the vehicle undergoing maintenance.
user_id	bigint	Foreign key. References the users table to identify who recorded the log.
date	date	The specific date when the maintenance was performed.
description	varchar(255)	Details of the maintenance work carried out (e.g., "Oil Change", "Tyre Replacement").
cost	decimal(8, 2)	The total cost incurred for the maintenance service.
created_at	timestamp	Timestamp indicating when the log entry was created in the system.
updated_at	timestamp	Timestamp indicating when the log entry was last modified.

5.5.10 Entity: notifications

Attribute Name	Type	Description
id	char(36)	Primary key. Unique identifier for notification
type	varchar(255)	Indicates the category of the notification such as distinguishing between a "Booking Approval" and a "Fine Issuance"
notifiable_type	varchar(255)	Identifies the type of user receiving the message

notifiable_id	bigint	The ID of the specific entity (user) receiving the notification.
data	text	JSON text containing the payload details of the notification (e.g., messages, links, related IDs).
read_at	timestamp	Timestamp indicating when the user read the notification; NULL if unread.
created_at	timestamp	Timestamp indicating when the notification was sent.
updated_at	timestamp	Timestamp indicating when the notification status was last updated such as marked as read.

5.5.11 Entity: password_reset_tokens

Attribute Name	Type	Description
email	varchar(60)	Primary key. The email address of the user requesting a password reset. Serves as the unique identifier for the request.
token	varchar(255)	A secure, hashed verification token generated by the system to validate the reset request.
created_at	timestamp	Timestamp indication when the reset request was created. Used to determine token expiration.

5.5.12 Entity: payments

Attribute Name	Type	Description
id	bigint	Primary key. Unique identifier for payment.
booking_id	bigint	Foreign key. References the bookings table to identify the associated rental transaction.
amount	decimal(8, 2)	The monetary value of the payment made.
payment_method	varchar(50)	The mode of payment (e.g., 'DuitNow QR', 'Manual Transfer').

receipt_image	varchar(255)	Stores the file path or URL to the uploaded proof of payment.
status	varchar(30)	Current status of the payment (e.g., 'Pending', 'Verified', 'Rejected').
created_at	timestamp	Timestamp indicating when the payment record was created.
updated_at	timestamp	Timestamp indicating when the payment status/details were last modified.

5.5.13 Entity: pricing_rules

Attribute Name	Type	Description
id	bigint	Primary key. Unique identifier for pricing rules
pricing_tier_id	bigint	Foreign key. References the pricing_tiers table to identify the associated pricing category.
hour_limit	int	The duration threshold (in hours) for this price point (e.g., 12 hours, 24 hours).
price	decimal(8, 2)	The rental cost associated with the specified hour limit.
created_at	timestamp	Timestamp indicating when this pricing rule was created.
updated_at	timestamp	Timestamp indicating when this pricing rule was last modified.

5.5.14 Entity: pricing_tiers

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for pricing tiers.
name	varchar(50)	The name of the pricing category (e.g., "Tier 1", "Tier 2"). Assigned to vehicles to determine their rate structure.
description	text	Optional text describing the details or eligibility of this tier.

created_at	timestamp	Timestamp indicating when the pricing tier was created.
updated_at	timestamp	Timestamp indicating when the tier details were last modified.

5.5.15 Entity: promotions

Attribute Name	Type	Description
id	bigint	Primary key. Unique identifier for promotions
code	varchar(30)	The alphanumeric code users enter to redeem the promotion (e.g., "WELCOME10").
description	varchar(255)	Details explaining the promotion's benefits.
discount_value	decimal(8, 2)	The numerical value of the discount (can be percentage or fixed amount).
type	varchar(20)	The calculator type for the discount (e.g., 'percent', 'fixed', 'free_hours').
start_date	date	The date when the promotion becomes valid.
end_date	date	The date when the promotion expires.
is_active	tinyint(1)	Boolean flag (0 or 1) indicating if the promotion is currently enabled.
created_at	timestamp	Timestamp indicating when the promotion was created.
updated_at	timestamp	Timestamp indicating when the promotion was last modified.

5.5.16 Entity: users

Attribute Name	Type	Description
id	bigint	Primary key. Unique identifier for users
name	varchar(100)	The full name of the user.
email	varchar(60)	The user's email address (used for login and notifications).

email_verified_at	timestamp	Timestamp indicating when the user verified their email address.
password	varchar(255)	The hashed password for account security.
role	varchar(15)	The user's access level (e.g., 'customer', 'staff', 'top manager').
matric_staff_id	varchar(15)	Unique institutional ID (Matric No. for students, Staff ID for staff).
matric_card_path	varchar(255)	Stores the file path to the uploaded matric card image.
nric_passport_path	varchar(255)	Stores the file path to the uploaded NRIC or Passport image.
nric_passport	varchar(255)	The text value of the NRIC or Passport number.
expired_date	date	The expiration date of the user's license.
driving_license_path	varchar(255)	Stores the file path to the uploaded driving license image.
phone_number	varchar(20)	The user's contact phone number.
address	text	The residential or mailing address of the user.
nationality	varchar(100)	The nationality of the user.
emergency_name	varchar(100)	Name of the customer's emergency contact person.
emergency_contact	varchar(20)	Phone number of the customer's emergency contact person.
emergency_relationship	varchar(20)	The relationship of the emergency contact to the customer (e.g., Parent, Sibling).
is_blacklisted	tinyint(1)	Boolean flag indicating if the customer is banned from accessing the system.
blacklist_reason	text	Textual reason explaining why the user was blacklisted.
college_id	bigint	Foreign key. References the colleges table.
faculty_id	bigint	Foreign key. Reference the faculties table

remember_token	varchar(100)	Token used for "Remember Me" session functionality.
created_at	timestamp	Timestamp indicating when the user account was registered.
updated_at	timestamp	Timestamp indicating when the user profile was last updated.
bank_name	varchar(255)	Name of the bank for refund purposes.
account_name	varchar(255)	The bank account number for refunds or deposit
account_holder	varchar(255)	Name of the account holder as per bank records.
salary	decimal(10, 2)	The salary amount (relevant for staff/manager for internal records).

5.5.17 Entity: user_vouchers

Attribute Name	Type	Description
id	bigint	Primary key. Unique auto incrementing identifier for user_vouchers records.
user_id	varchar(15)	Foreign key. References the users table(matric_staff_id) to identify the voucher owner.
voucher_id	bigint	Foreign key. References the vouchers table to identify the specific voucher type
used_at	timestamp	Timestamp indicating when a voucher was applied to a booking. NULL if unused.
created_at	timestamp	Timestamp when the voucher was claimed by or assigned to the user.
updated_at	timestamp	Timestamp when the record was last modified such as marked as used.

5.5.18 Entity: vehicles

Attribute Name	Type	Description
----------------	------	-------------

id	bigint	Primary key. Unique auto incrementing identifier for each vehicle.
vehicle_id_custom	varchar(10)	Custom human readable identifier such as 'V001' used for internal fleet tracking
plate_number	varchar(10)	Official vehicle registration number. Unique system-wide.
brand	varchar(255)	Manufacturer of the vehicle such as 'Perodua'
model	varchar(255)	Specific model name of the vehicle such as 'Myvi'
year	int	Manufacturing year of the vehicle
capacity	int	Maximum number of passengers the vehicle can legally carry
price_per_hour	decimal(8, 2)	Base hourly rental rate. Used as a fallback if no specific tier is set because the price provided by stakeholder is 1 hour, 3 hour and so on
vehicle_image	varchar(255)	File path to the uploaded image of the vehicle and save as URL
is_hasta_owned	tinyint(1)	Boolean flag 1 if the car is owned by Hasta, 0 if it is a third party rental
owner_name	varchar(100)	Name of the car owner
owner_ic_path	varchar(255)	File path to the uploaded IC copy of the car owner and save as URL
owner_license_path	varchar(255)	File path to the uploaded driving license copy of the car owner and save as URL
geran_path	varchar(255)	File path to the uploaded vehicle registration card and save as URL
insurance_cover_path	varchar(255)	File path to the insurance cover note document and save as URL
current_fuel_bar	int	Current fuel level recorded for tracking

status	varchar(30)	Current operational status. Values: 'Available', 'Rented', 'Maintenance', 'Unavailable'
type_id	int	Identifier linking to vehicle type category
road_tax_expiry	date	Expiration date of the vehicle's road tax
insurance_expiry	date	Expiration date of the vehicle insurance policy.
created_at	timestamp	Timestamp when the vehicle record was created
updated_at	timestamp	Timestamp when the vehicle details were last modified
pricing_tier_id	bigint	Foreign key links to pricing_tiers table

5.5.19 Entity: vehicle_types

Attribute Name	Type	Description
id	bigint	Primary key. Unique identifier for vehicle type
code	int	System code. Maps to the selection value in dropdown menus such as 1 for Compact, 2 for Sedan
name	varchar(30)	Type Name such as 'Compact', 'Sedan'
base_price	decimal(8, 2)	Legacy field designated for base pricing. Retained for backward compatibility or future implementation, superseded by specific vehicle pricing rules.
created_at	timestamp	Timestamp when the record was created
updated_at	timestamp	Timestamp when the record was updated

5.5.20 Entity: vouchers

Attribute Name	Type	Description
id	bigint	Primary key. Unique identifier for voucher
user_id	varchar(15)	Foreign key links to users table(matric_staff_id). Nullable. If set, the voucher is exclusive to that specific user; if NULL, it is a public voucher
code	varchar(30)	Voucher code. The unique alphanumeric code users enter to redeem
name	varchar(50)	Display name of voucher. A simple descriptive label for the voucher
type	varchar(20)	Reward type. Logic determinant: 'fixed'(RM) , 'percent'(%), or free_hours
value	decimal(8, 2)	Reward value. Numerical amount corresponding to the type
is_active	tinyint(1)	Status flag. 1 indicates it is active or redeemable. Disabled or invalidated can be set by the admin.
expires_at	timestamp	Expiration timestamp. Nullable. If set, the voucher cannot be used after this date.
created_at	timestamp	Timestamp when the voucher was created
updated_at	timestamp	Timestamp when the voucher details were last modified.
single_use	tinyint(1)	One time use the flag. If 1, limit each unique user to redeeming this code only once
uses_remaining	int	Global usage counter. Nullable. Decrements on each use. If verified as greater than 0, limit total number of redemptions system-wide.

6.0 Summary

This phase focuses on the database conceptual design for the Hasta Car Rental System to replace the existing manual workflow. By analysing the business processes using Data Flow Diagrams (DFD), the main data interactions between customers, staff, and vehicle operations were identified. This helps ensure that the proposed database structure reflects the actual operational needs of the company.

Based on the identified business rules, the Entity-Relationship Diagram (ERD) and Enhanced Entity-Relationship Diagram (EERD) were developed to organise the data into a structured relational model. The design includes a unified user structure to manage different user roles, supports both company-owned and third-party vehicles, and covers the full rental process from booking and payment to inspection and penalty handling.

Overall, this database conceptual design provides a clear and structured basis for the subsequent logical and physical database design phases. It addresses the key data management issues found in the current system and prepares the project for further implementation.

7.0 Appendice

7.1 Use Case Diagram for the System

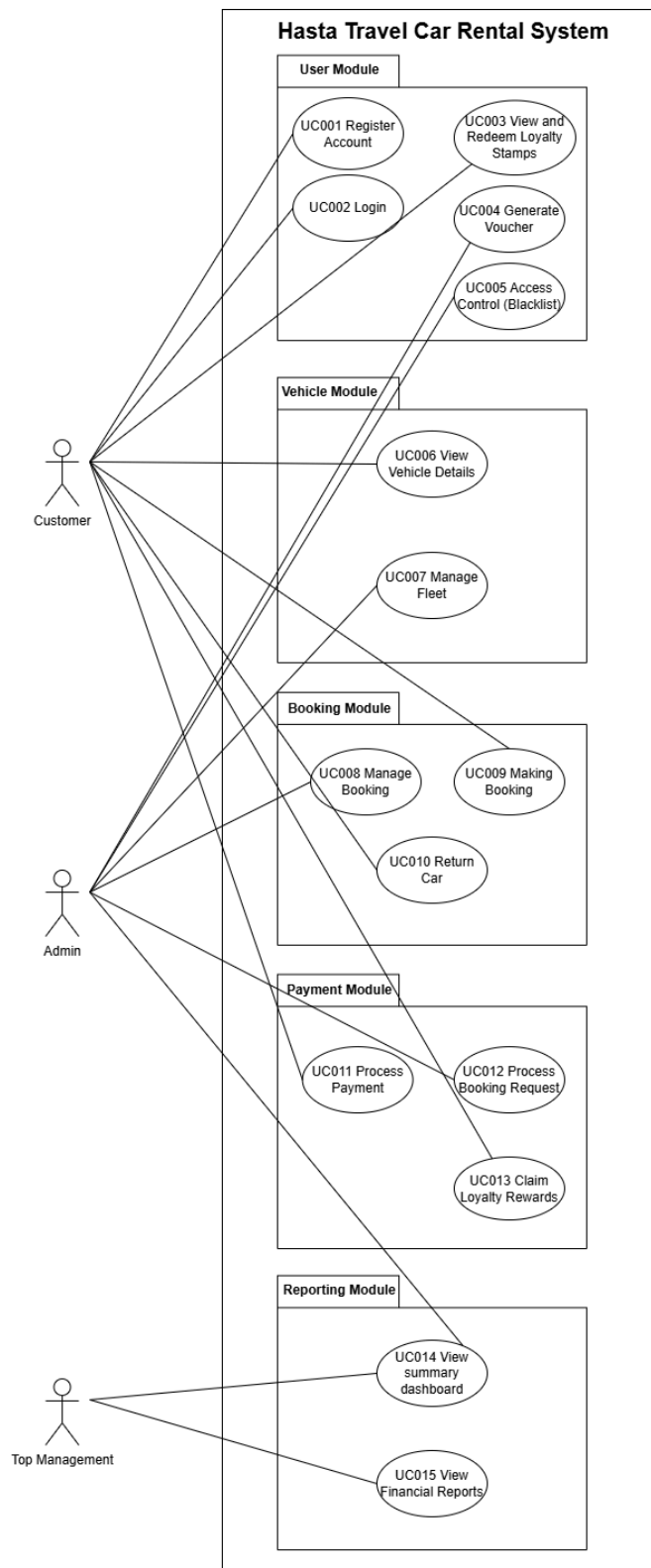


Figure 4.1 Use Case Diagram for <Hasta Travel Car Rental System>

Table 4.1: Description of Module and Functions for <Hasta Travel Car Rental System>

Use case	Function	Description
UC001	Register Account	This use case allows customers to create a personal account by providing their personal and academic information to access rental services.
UC002	Login	This use case allows customers to securely login into the system by using their credentials to access their personalized dashboard.
UC003	View and Redeem Loyalty Stamps	This use case allows customers to view their current amount of the loyalty stamps so they can determine how many stamps they still need to collect to redeem a reward.
UC004	Generate Voucher	This use case allows admins to generate unique digital voucher codes after a successful redemption or promotion setup to ensure customers can apply specific discounts to their future bookings.
UC005	Access Control (Blacklist)	This use case allows admins to restrict access for problematic users to the system to prevent high-risk renters damaging the vehicles or causing operational delays.
UC006	View Vehicle Details	This use case allows customers to view vehicle details information in the system, including the car availability, specifications and prices.
UC007	Manage Fleet	This use case allows admin to manage vehicle records, track maintenance and monitor document expiry to ensure the vehicle is accurate, maintained and ready for customers.
UC008	Manage Booking	This use case allows admin to manage all booking records to ensure that they can approve, reject, or update bookings efficiently.

UC009	Making Booking	This use case allows customers to make a booking for a vehicle and reserve for their desired booking date and time.
UC010	Return Car	This use case allows customers to return a rented car and provide condition details to formally complete the booking process.
UC011	Process Payment	This use case allows customers to complete the checkout process smoothly by selecting a pickup location, applying valid vouchers and uploading the payment receipt directly to secure the booking immediately.
UC012	Process Booking Request	This use case allows admins to access and view the payment receipt for a booking to ensure they can verify the transaction before releasing the vehicle to the customer.
UC013	Claim Loyalty Rewards	This use case allows customers to select and apply previously redeemed voucher during the payment stage to ensure they can apply the discount and reduce the total rental cost.
UC014	View Summary Dashboard	This use case allows top management to view a high-level dashboard with visual analytics to quickly assess the company's financial status and customer distribution.
UC015	View Financial Reports	This use case allows top management to generate financial reports for specific time periods to track profitability over time.

7.2 Activity Diagram for the System

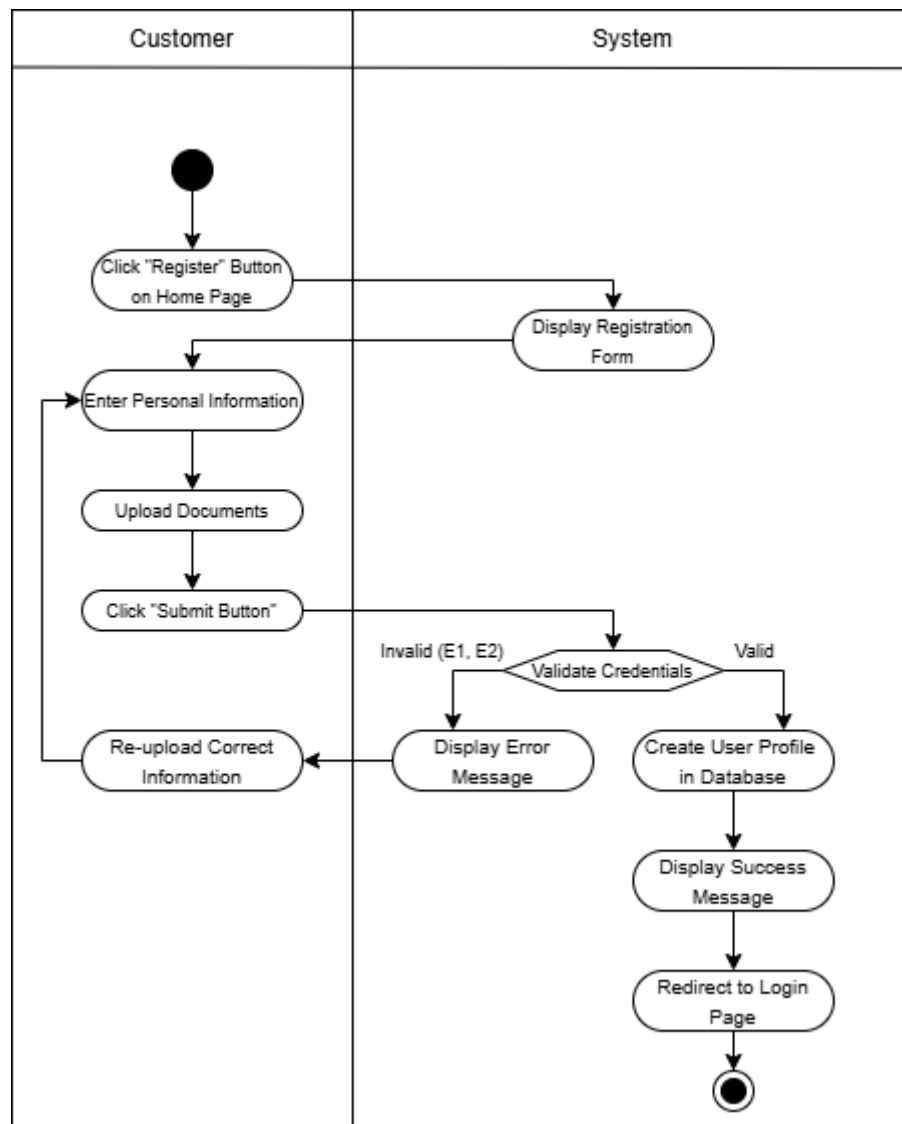


Figure 7.2.1: Activity Diagram for <Register Account>

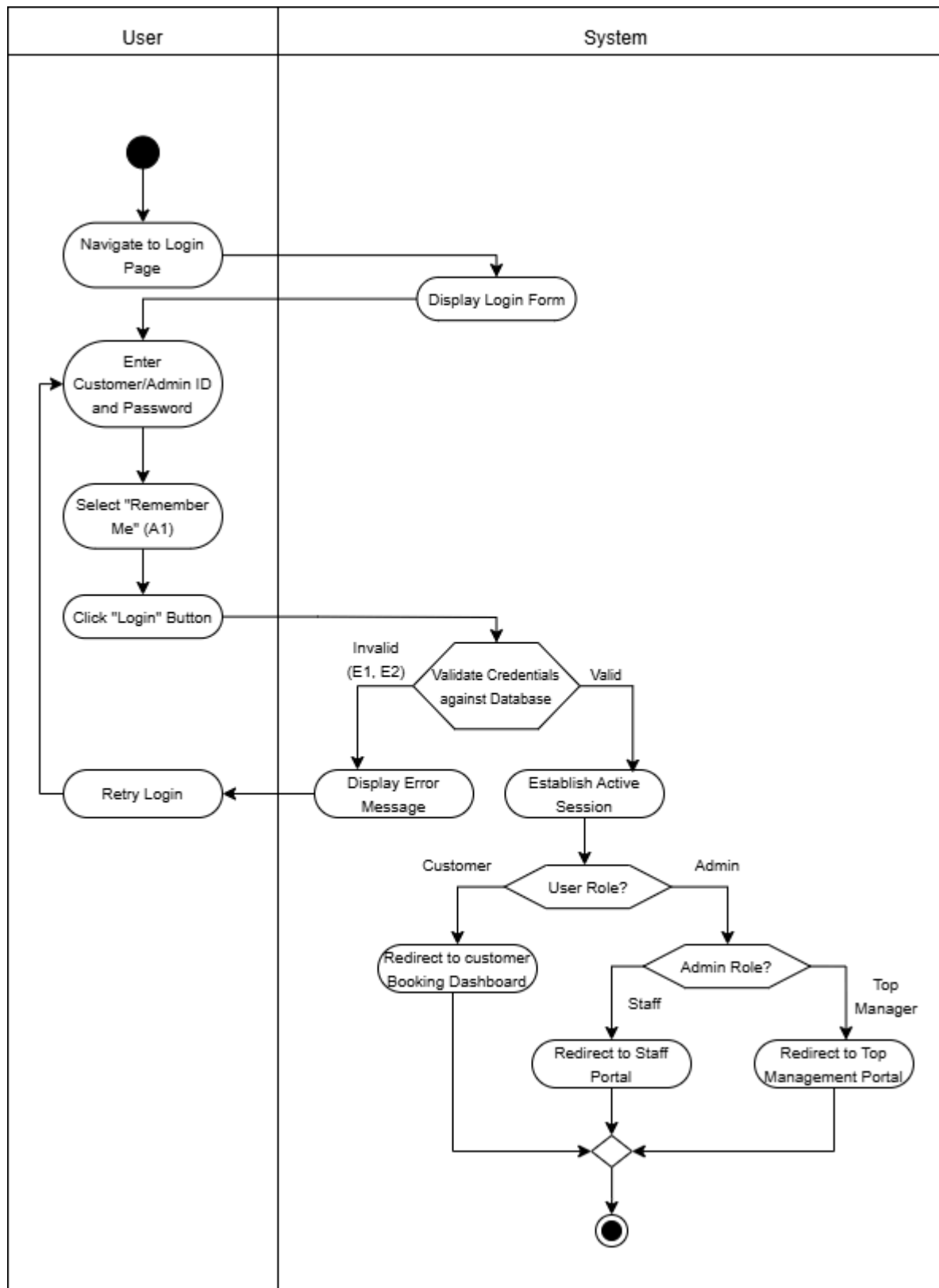


Figure 7.2.2: Activity Diagram for <Login>

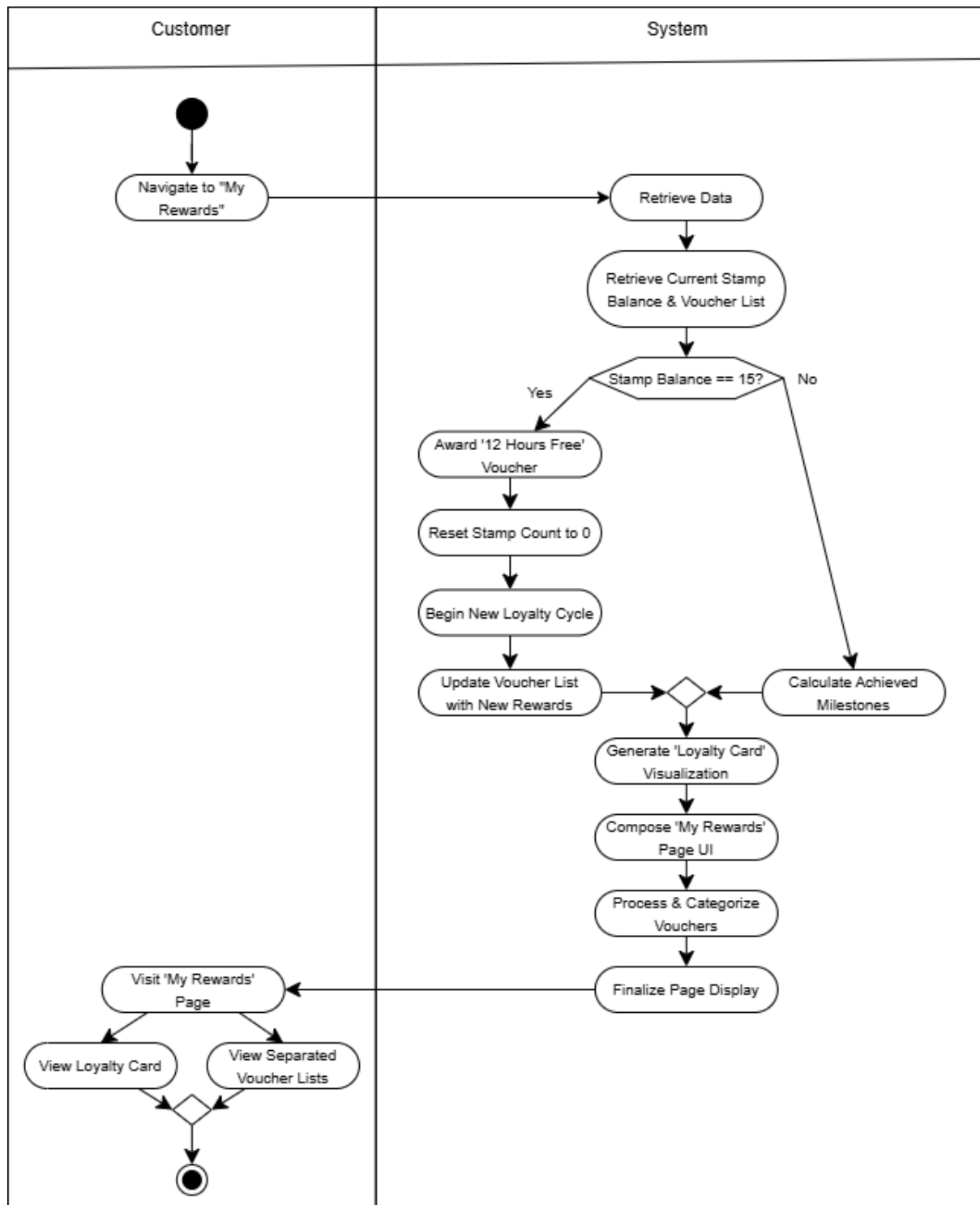


Figure 7.2.3: Activity Diagram for <View and Redeem Loyalty Stamps>

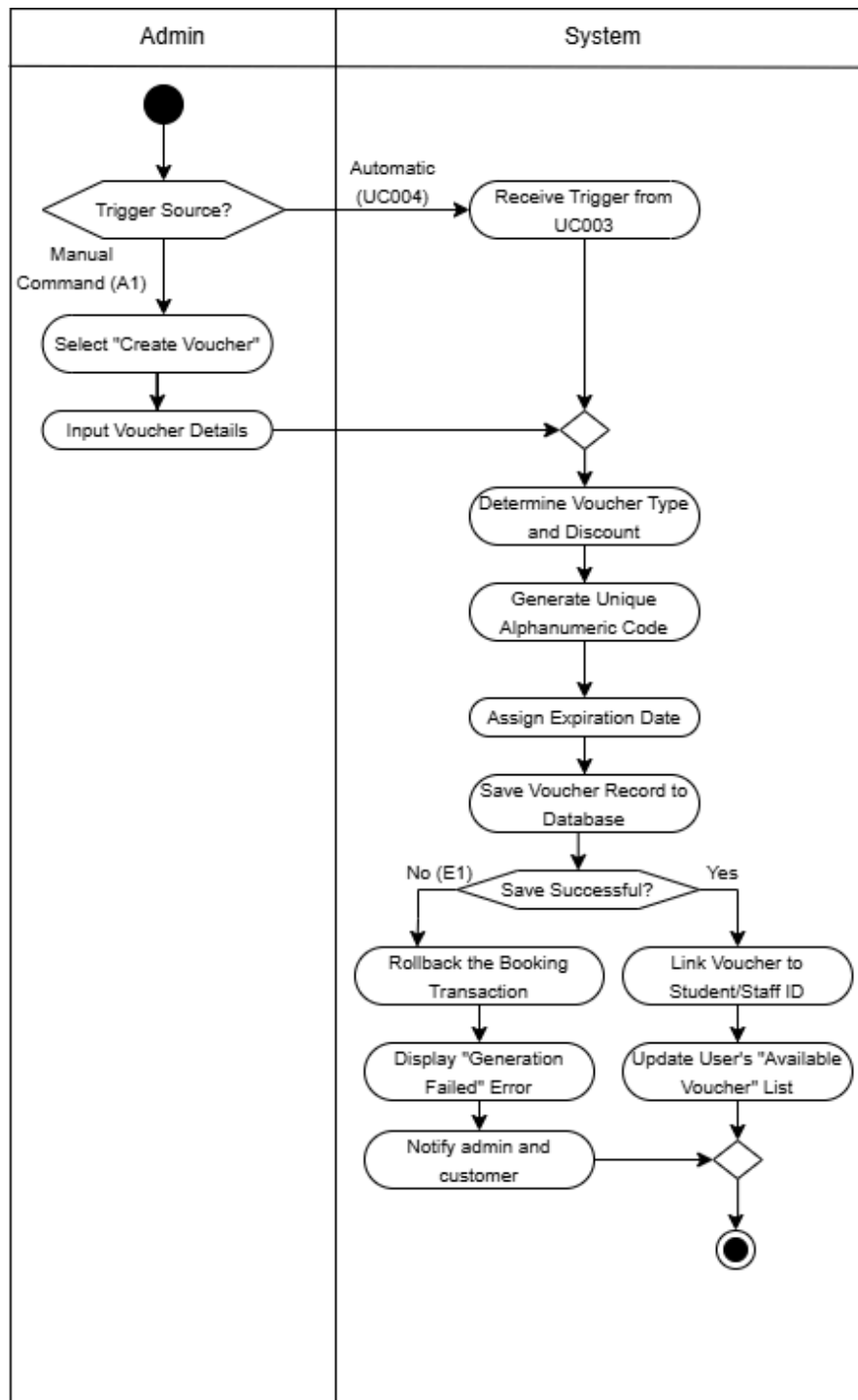


Figure 7.2.4: Activity Diagram for <Generate Voucher>

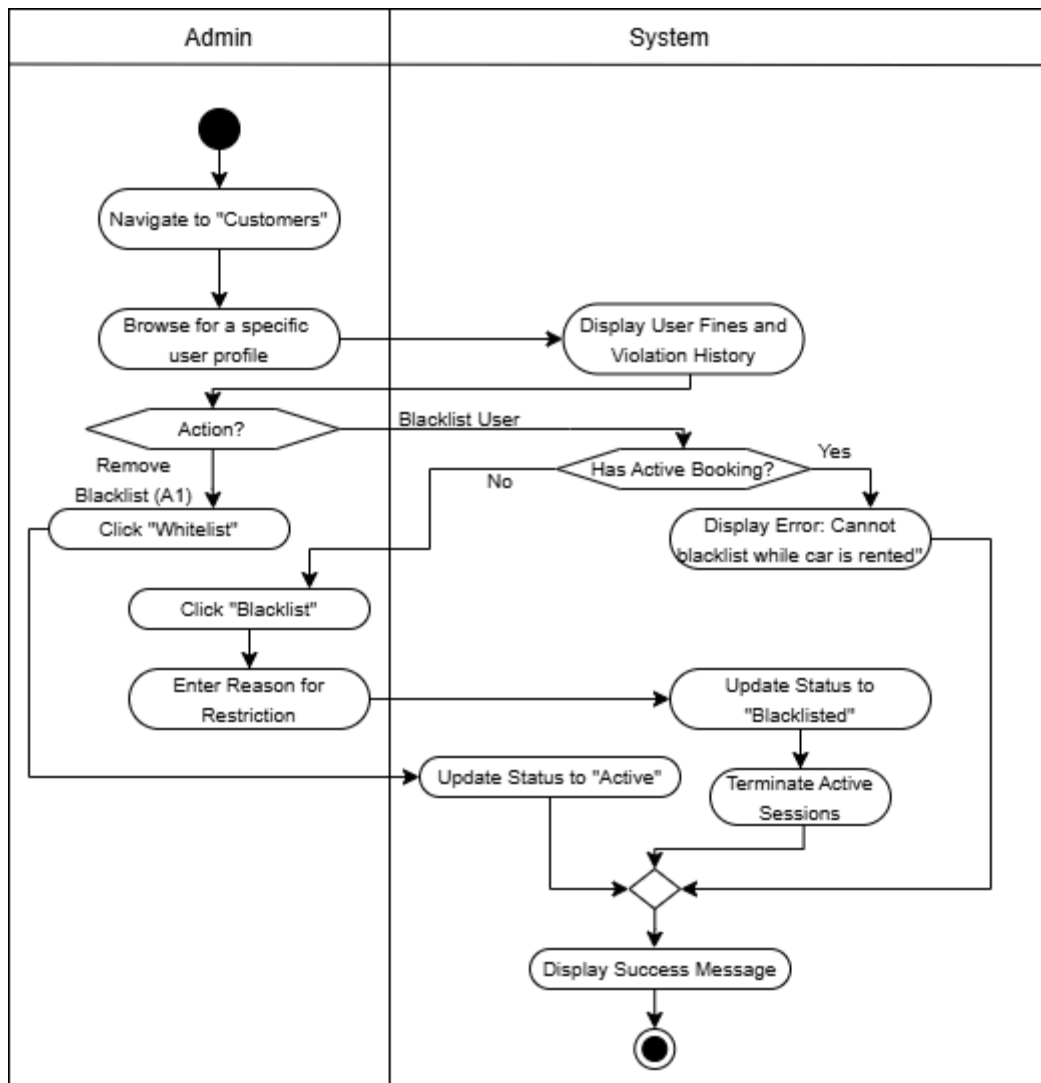


Figure 7.2.5: Activity Diagram for <Access Control (Blacklist)>

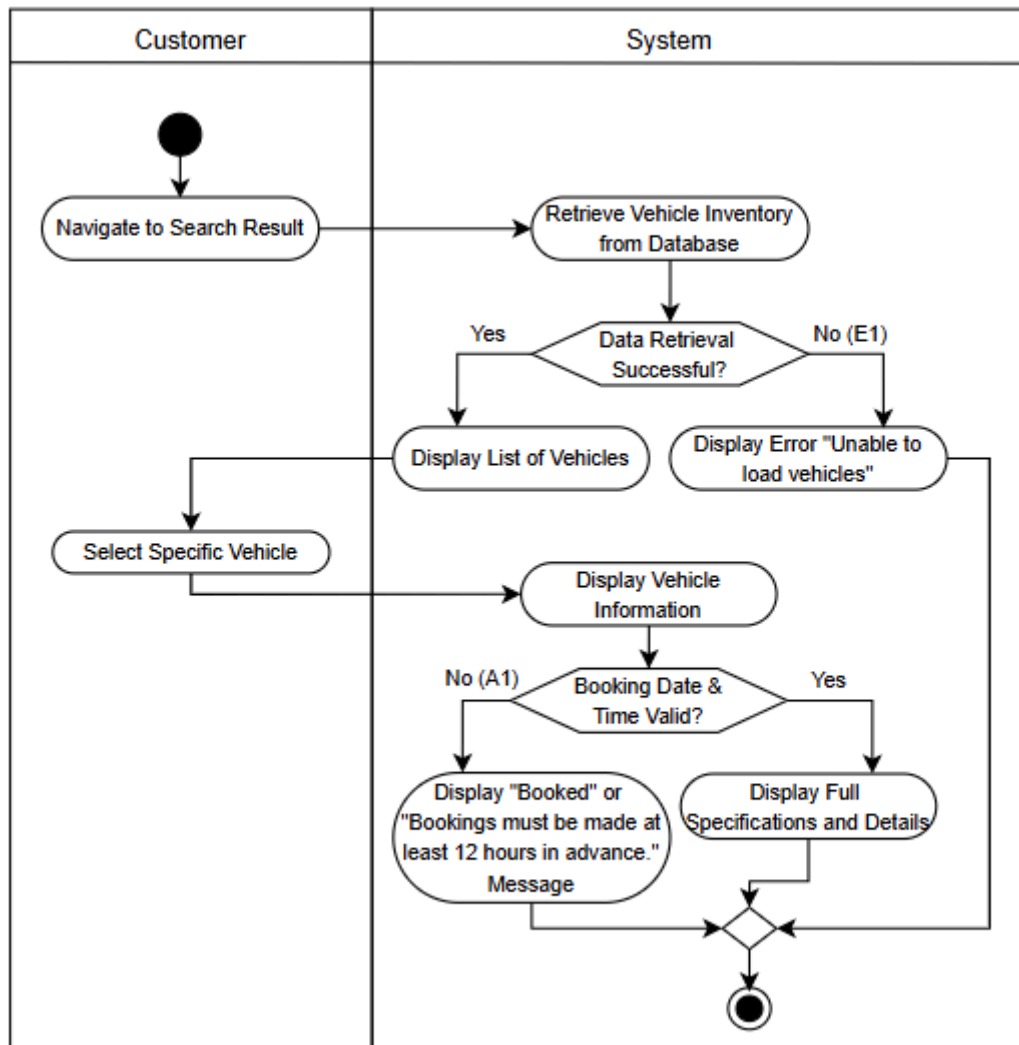


Figure 7.2.6: Activity Diagram for <View Vehicle Details>

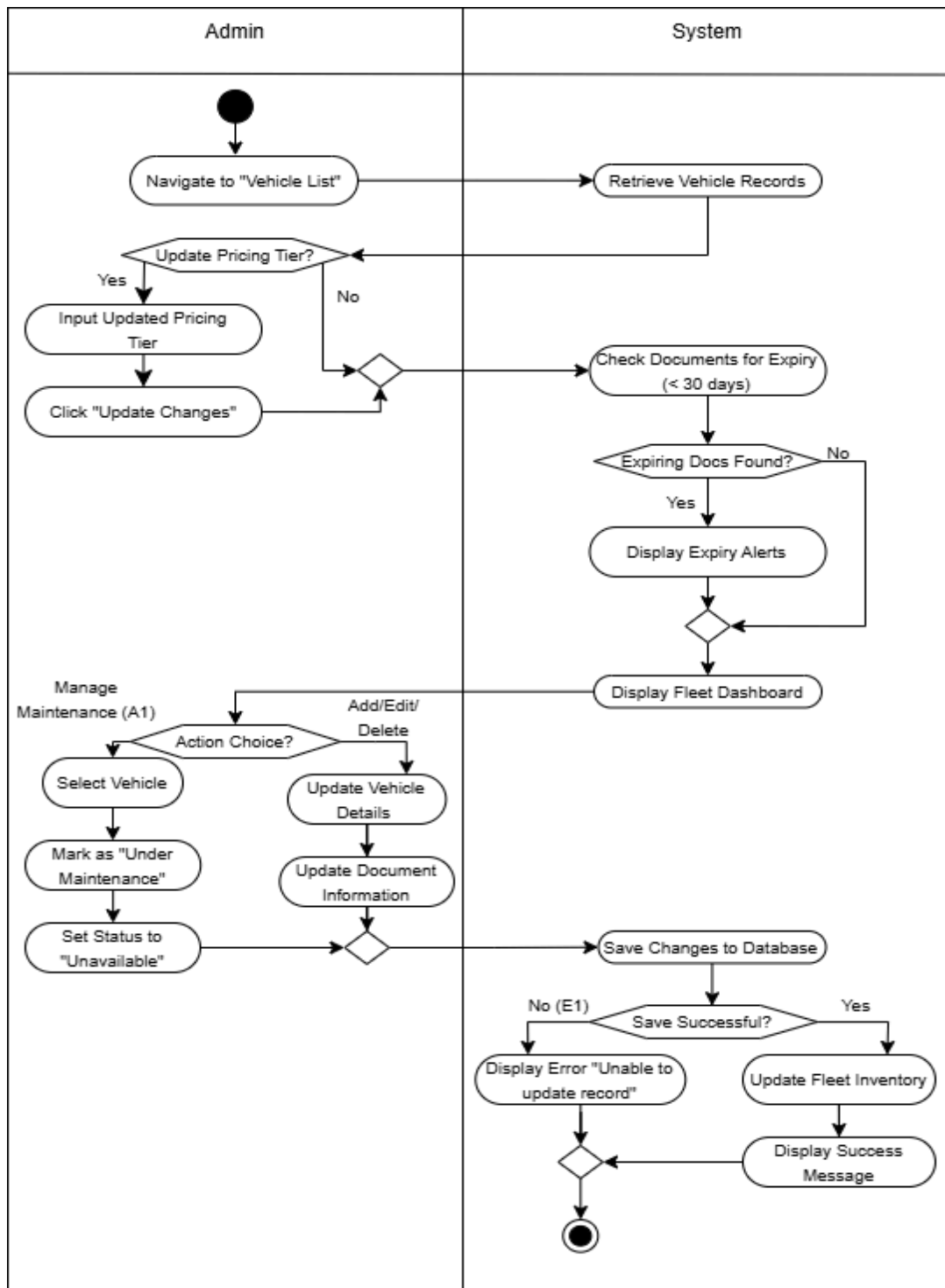


Figure 7.2.7: Activity Diagram for <Manage Fleet>

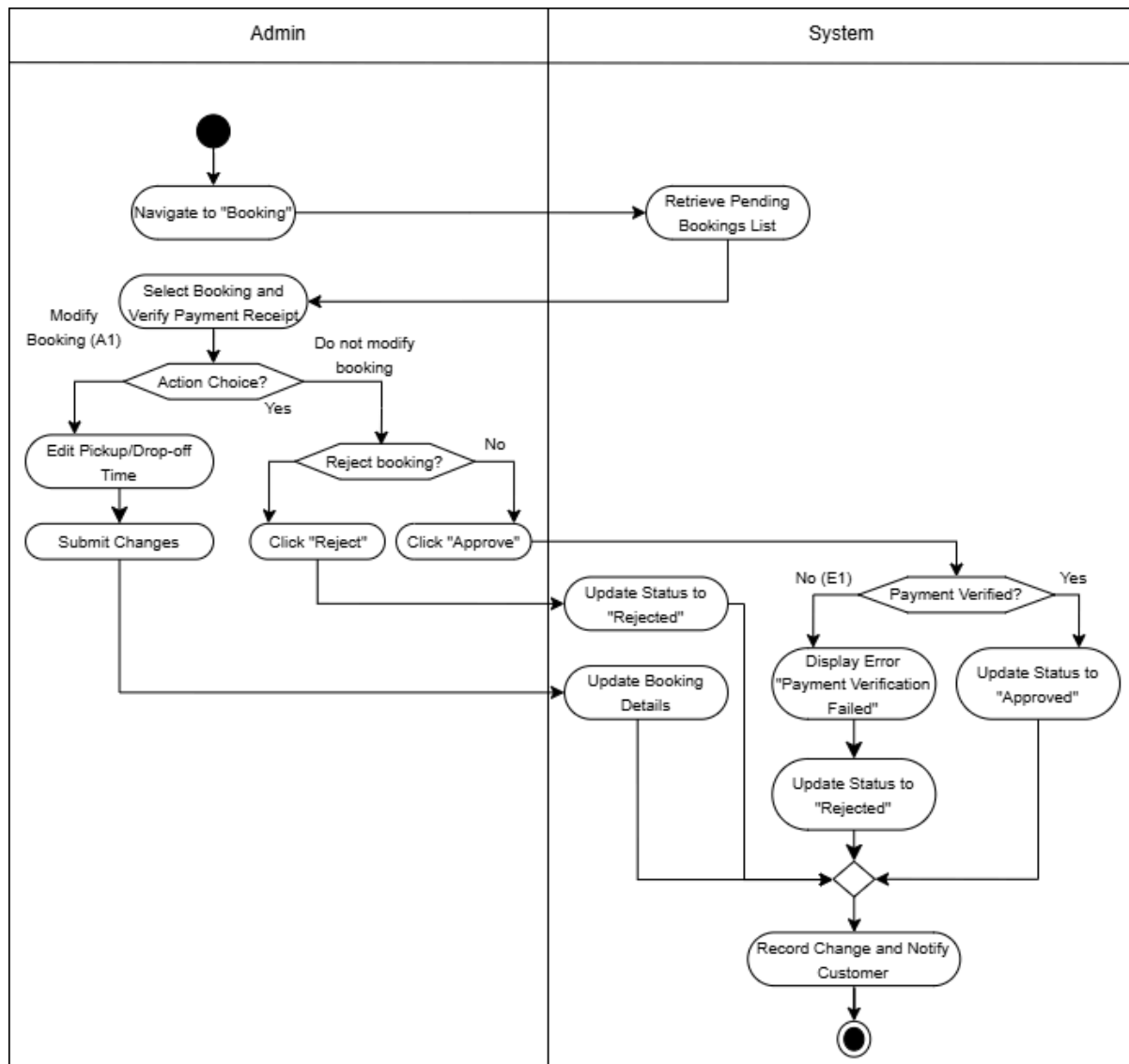


Figure 7.2.8: Activity Diagram for <Manage Booking>

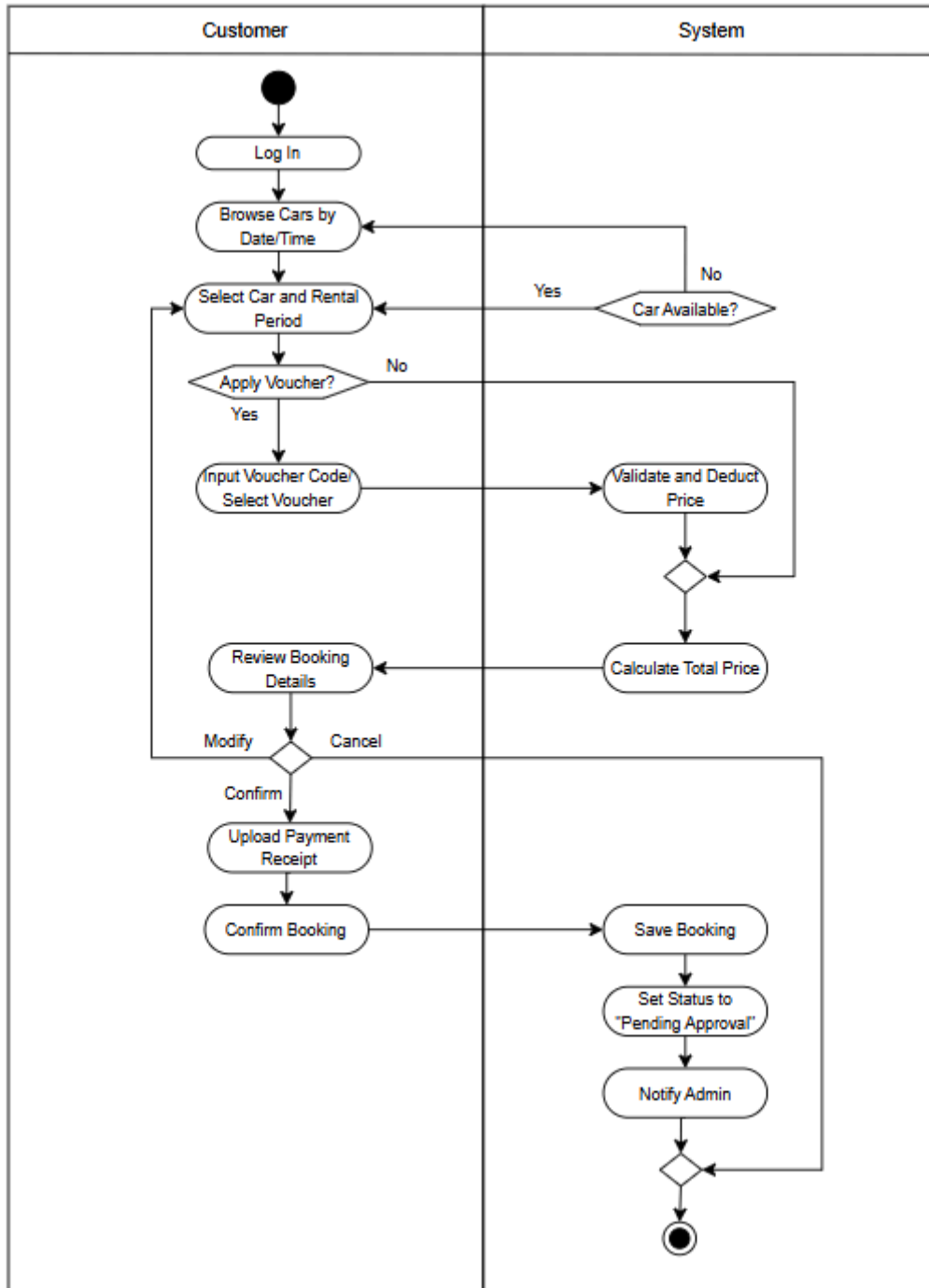


Figure 7.2.9: Activity Diagram for <Making Booking>

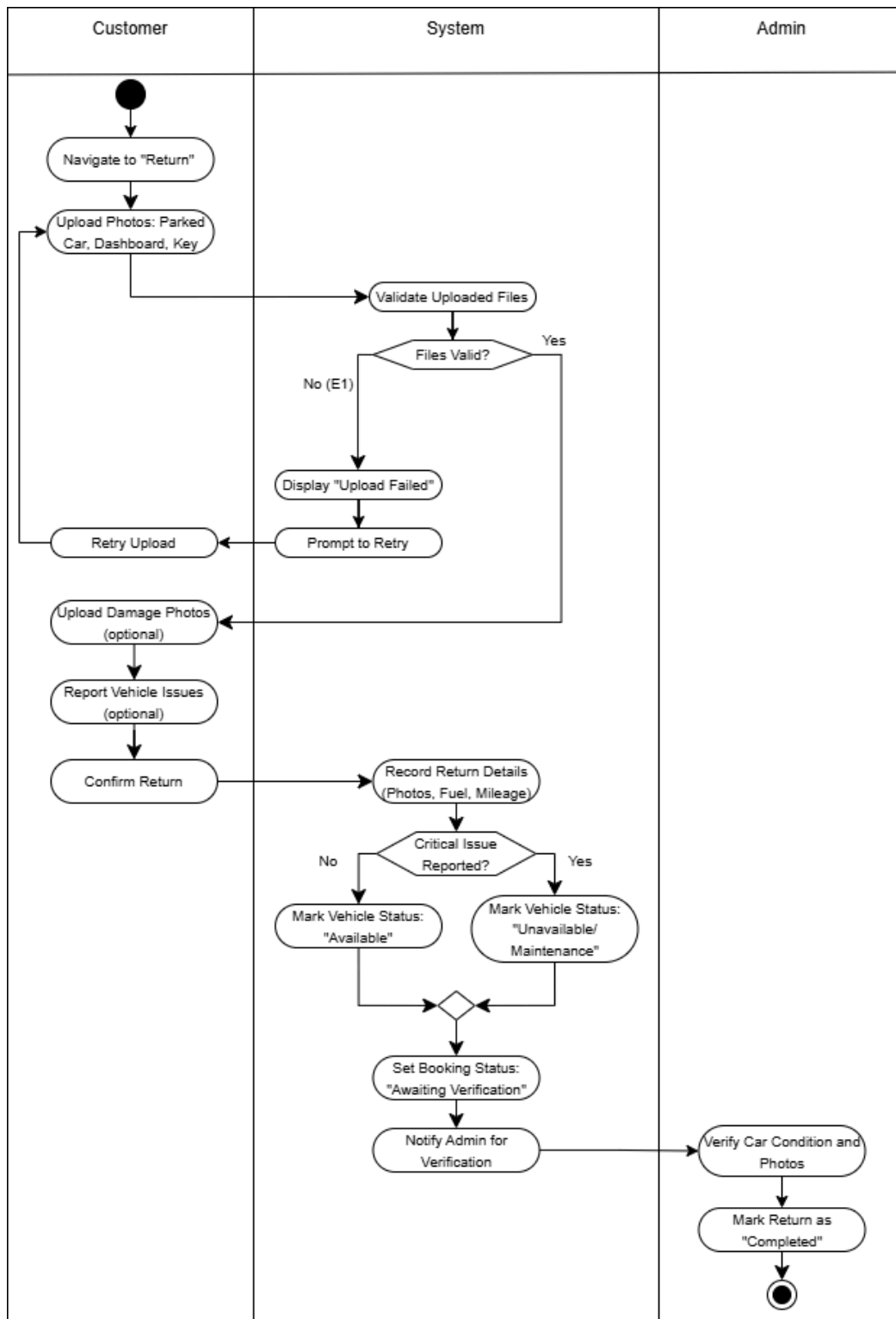


Figure 7.2.10: Activity Diagram for <Return Car>

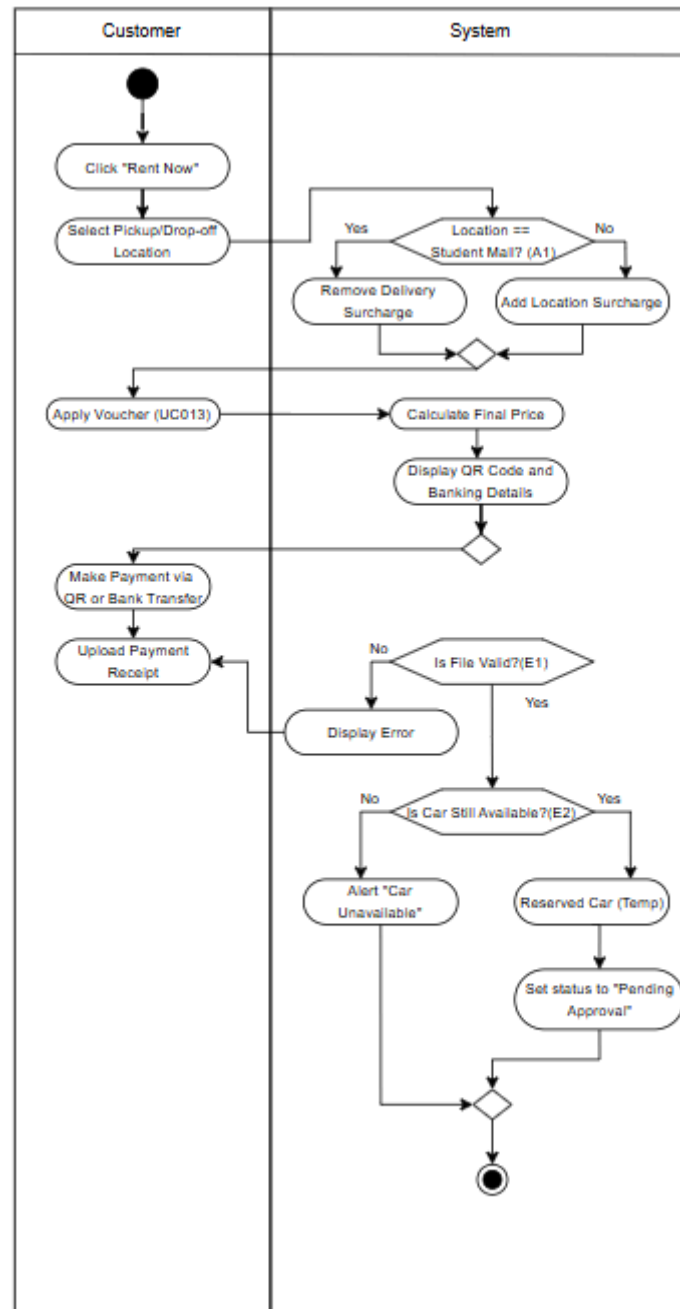


Figure 7.2.11: Activity Diagram for <Process Payment>

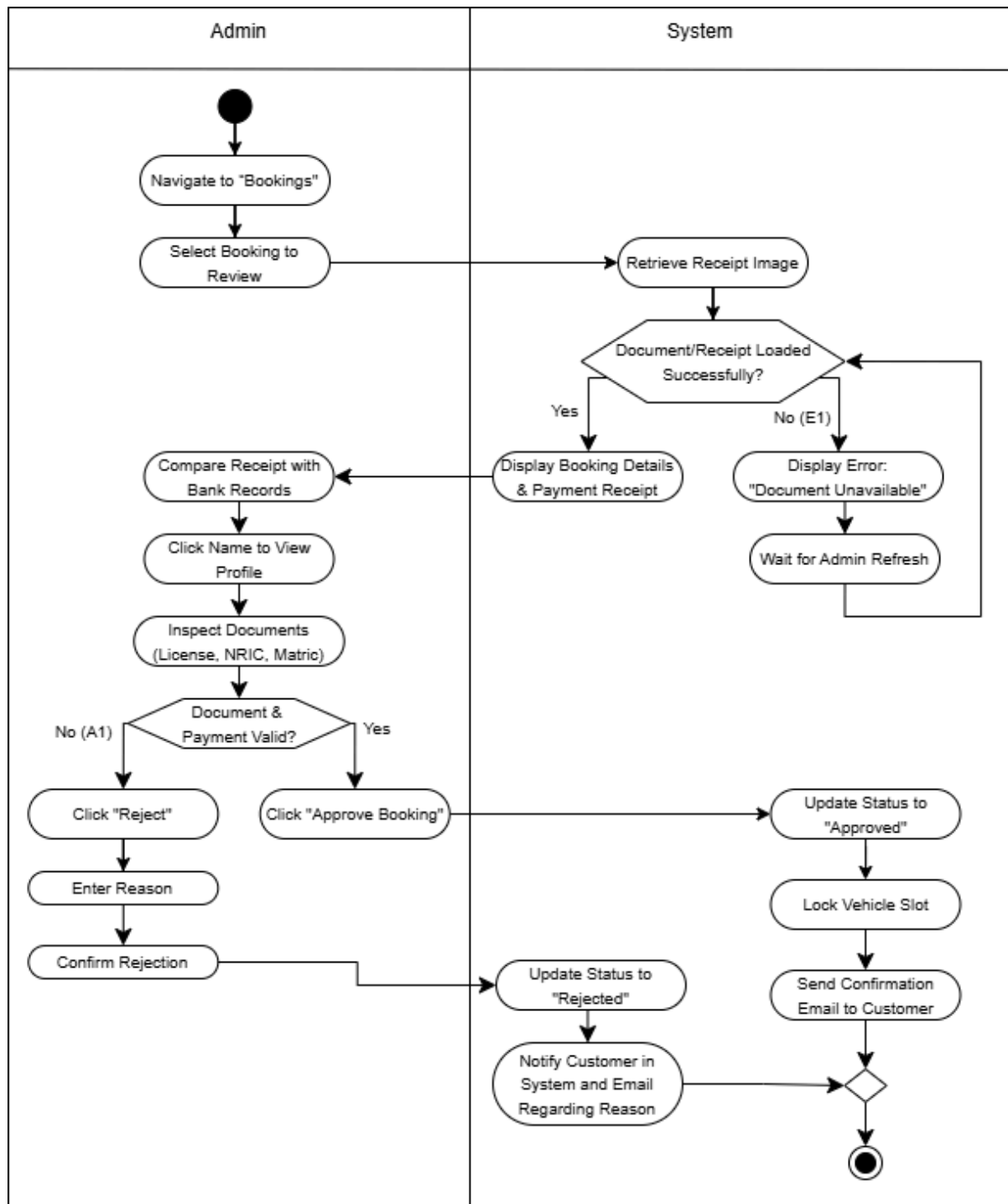


Figure 7.2.12: Activity Diagram for <Process Booking Request>

UC013

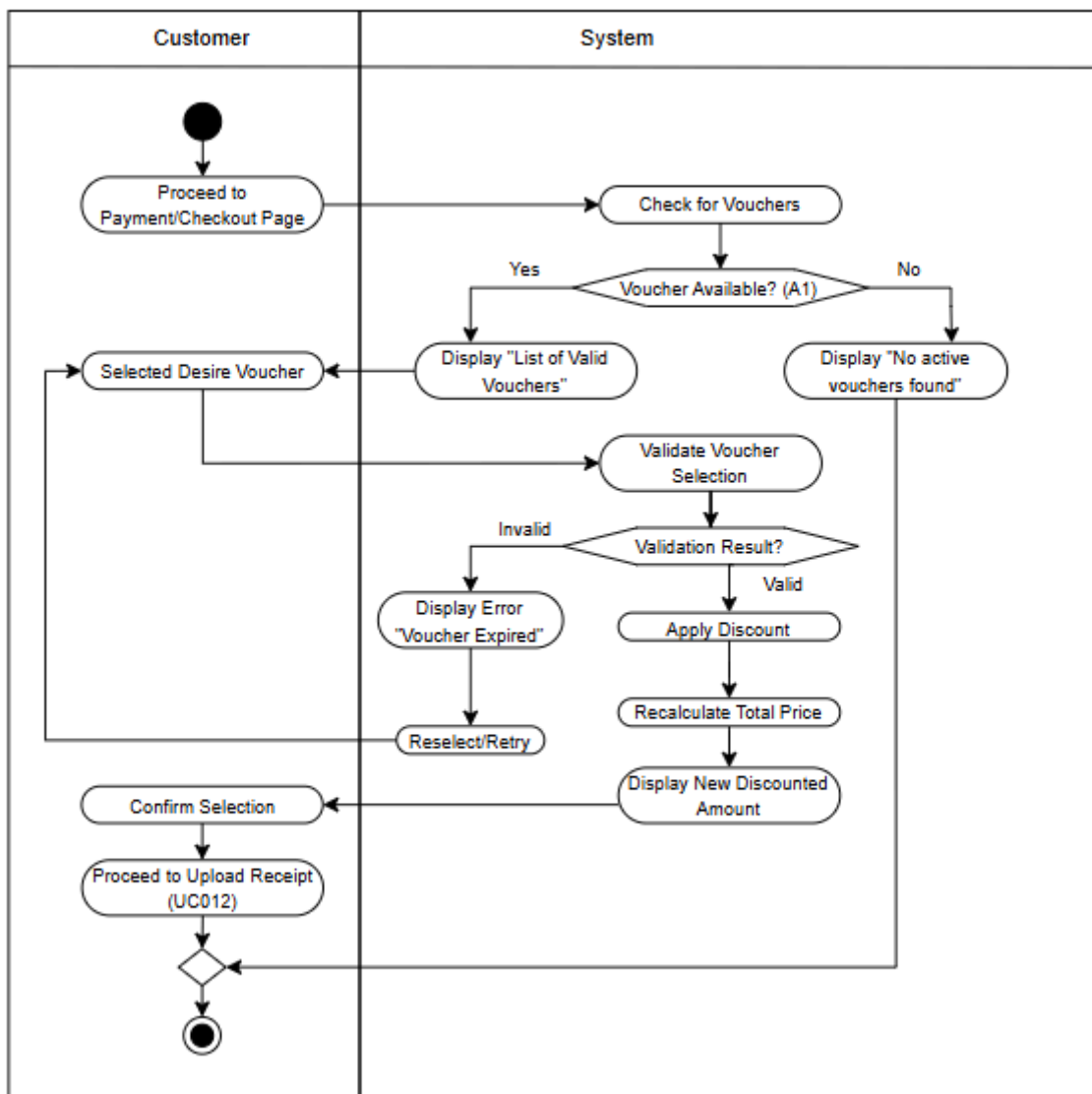


Figure 7.2.13: Activity Diagram for <Claim Royalty Rewards>

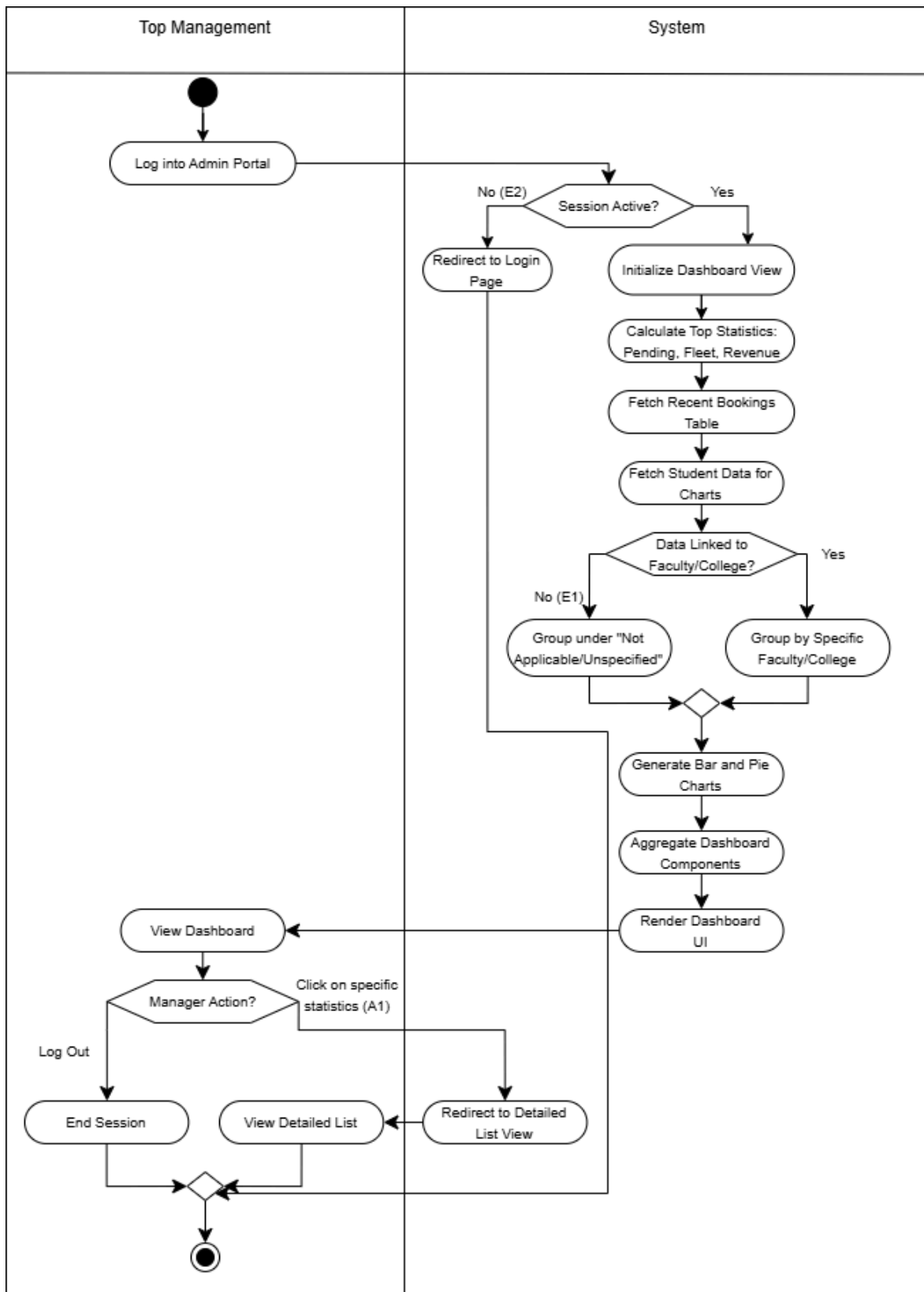


Figure 7.2.14: Activity Diagram for <View Summary Dashboard>

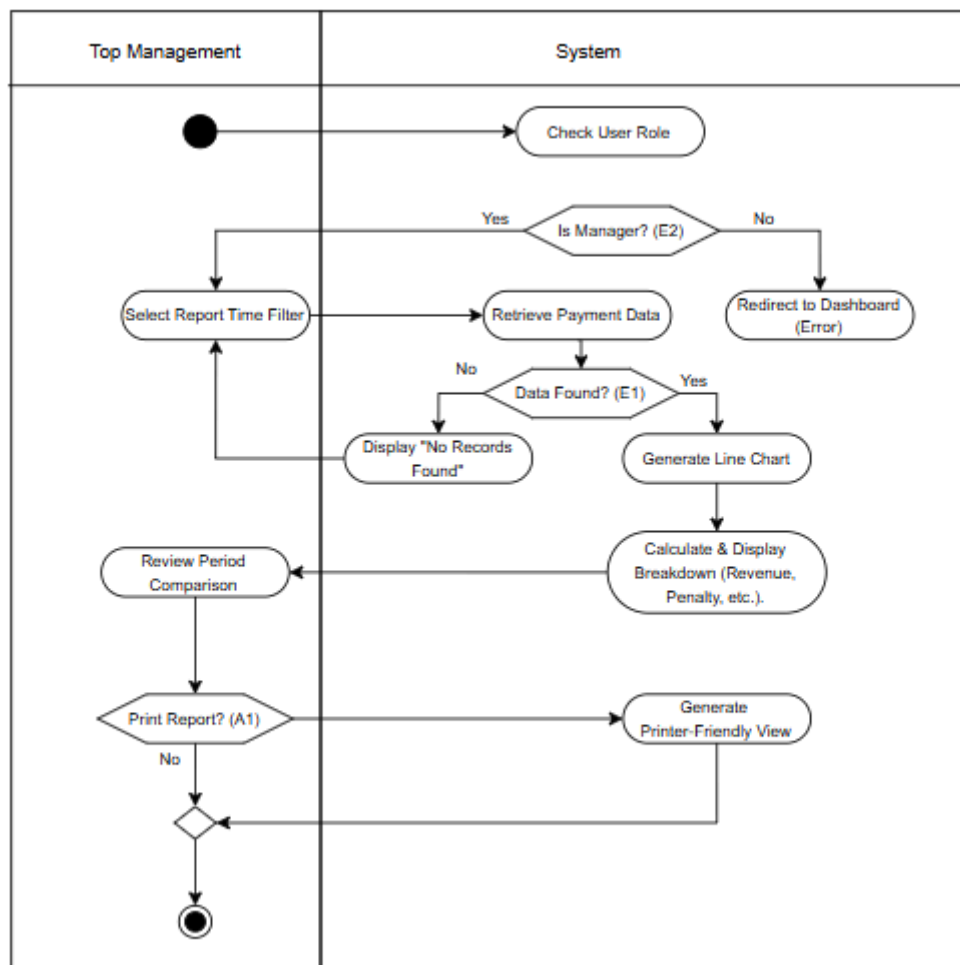


Figure 7.2.15 Activity Diagram for <View Financial Reports>

7.3 Meeting Log

Meeting Title	Discuss Database P2 Report
Date/Time	5 December 2025 6.00 p.m.
Location	Google Meet
Meeting MC	Gwee Zi Ni
Attendance	1. Gwee Zi Ni 2. Kalaitharan A/L Palanyvelu 3. Lee Jia Yee 4. Michelle Ho Chia Xin 5. Tay Xin Ying
Agenda	1. Discussion on the “To-Be” Data Flow Diagram (DFD) - The team discussed the “To-Be” DFD to illustrate how data will flow in the proposed system. Improvements were suggested to ensure the processes, data stores, and external entities were clearly represented.
	2. Proposed business rules and data & transactions -The proposed business rules were reviewed to ensure they align with system requirements. The team also discussed the types of data involved and how transactions would be processed within the system.
	3. ERD and EERD development - Methods to generate the ERD and EERD were discussed. The main entities were identified, and relationships among entities were clarified to ensure accurate representation of the database structure.
	4. Data Dictionary - The team discussed how to create the Data Dictionary, focusing on defining tables, attributes, data types, and constraints for each field in the database.
	5. Task Distribution - Tasks were distributed among team

	members to ensure efficient completion of the Database P2 Report.
--	---

Boundaries of Database Application

The boundary of the database application covers all data storage and management required by the User, Vehicle, Booking, Payment, and Reporting modules as illustrated in the use case diagram. The database supports operations such as user registration and authentication, vehicle information management, booking creation and updates, payment and deposit tracking, loyalty reward processing, and report generation.

External services such as online payment gateways, notification delivery mechanisms, and user interface rendering are outside the database boundary. These components interact with the database only through stored transaction records, references, and status updates.

Major User Views

The database supports three major user views as defined in the use case diagram. Customers interact with the database to manage their bookings, payments, and personal information. Administrative staff access operational data related to vehicles, bookings, payments, and blacklist management. Top management users access summarized and aggregated data through dashboards and financial reports for monitoring and decision-making purposes.

Each user view is implemented through role-based access control to ensure data confidentiality and operational security.

Bibliography

Connolly, T. M., & Begg, C. E. (2015). *Database Systems : a Practical Approach to design, implementation, and Management*. Pearson.

Kendall, K. E., & Kendall, J. E. (2020). *Systems analysis and design*. Pearson.

Khan, R. A., & A Agrawal. (2014). *Software engineering a practitioner's approach*. Oxford Alpha Science International.

PHP. (2019). *PHP: Documentation*. php.net. <https://www.php.net/docs.php>

Visual Paradigm Online. (n.d.). Data Modeling: Conceptual vs Logical vs Physical Data Model.
<https://online.visual-paradigm.com/knowledge/visual-modeling/conceptual-vs-logical-vs-physical-data-model>

w3schools. (n.d.). *MySQL Introduction*. www.w3schools.com.
https://www.w3schools.com/mysql/mysql_intro.asp